

Appunti di Basi di dati 1

Emanuele Romano

Indice

1	Progettazione Concettuale	1
1.1	Progettazione Concettuale con Class Diagrams	1
1.1.1	Ruoli nelle Associazioni	2
1.1.2	Navigabilità e Direzionalità	2
1.1.3	Tipi Enumerativi (Enumerazioni)	3
1.1.4	Classi di Associazione	3
1.1.5	Grado di un'Associazione e Relazioni Ternarie	5
1.1.6	Aggregazione e Composizione	7
1.1.7	Generalizzazioni/Specializzazioni	8
1.2	La Documentazione di Supporto: I Dizionari dei Dati	9
2	Modello Relazionale e Progettazione Logica	11
2.1	Introduzione	11
2.2	Concetti Fondamentali	11
2.3	Le Chiavi	13
2.4	Traduzione delle Associazioni	14
2.4.1	Associazioni 1 a 1	14
2.4.2	Associazioni 1 a Molti (1:N)	16
2.4.3	Associazioni Molti a Molti (N:N)	17
2.5	Ristrutturazione dello Schema Concettuale	18
2.5.1	Analisi delle Chiavi: Chiavi Naturali vs Surrogate	19
2.5.2	Analisi degli Attributi Derivati e delle Ridondanze	20
2.5.3	Analisi delle Associazioni Uno a Uno: Fusione vs Separazione	21
2.5.4	Analisi degli Attributi Strutturati	22
2.5.5	Analisi degli Attributi a Valore Multiplo	24
2.5.6	Codifica delle Gerarchie (Ereditarietà)	25
2.6	Riepilogo: Il Ciclo di Vita della Progettazione dei Dati	26
3	Introduzione a SQL e Definizione dei Metadati	28
3.1	Nozioni introduttive	28
3.2	I Domini di Base (Tipi di Dato) nello Standard SQL	29
3.3	Operatori Logici e di Confronto	31
3.4	Istruzioni DDL	32
3.4.1	Creazione di uno schema logico	32
3.4.2	Creazione di una tabella	33
3.4.3	I Vincoli Intra-attributo (o in-line)	33
3.4.4	I Vincoli di Tabella (Out-of-line)	35
3.4.5	Classificazione Concettuale dei Vincoli	36
3.4.6	Integrità Referenziale e Chiavi Esterne (FOREIGN KEY)	38

4	SQL e Algebra Relazionale	41
4.1	Proiezioni	42
4.1.1	La Proiezione in SQL: Il comando SELECT	43
4.2	Selezioni	43
4.2.1	La Selezione in SQL: La clausola WHERE	44
4.2.2	Attributi Parziali e Logica Ternaria	45
4.3	Composizione di Proiezioni e Selezioni	46
4.3.1	La Composizione in SQL e l'Ordine Logico di Esecuzione	47
4.4	Qualificazione degli Attributi	48
4.5	Ridenominazione	48
4.5.1	La Ridenominazione in SQL: L'operatore AS	48
4.6	Operazioni Insiemistiche	49
4.6.1	Le Operazioni Insiemistiche in SQL	51
4.7	Il Prodotto Cartesiano e le Giunzioni (Join)	51
4.7.1	I Join in SQL	53
4.8	Giunzioni Esterne (Outer Join)	55
4.8.1	Le Giunzioni Esterne in SQL	56
4.8.2	Il pattern "Anti-Join": Isolare le assenze	56
A	Bozze	58
A.1	parti di lezioni da rivedere	58
A.2	La Giunzione Naturale in SQL: NATURAL JOIN e USING (da inserire in appendice)	58

Progettazione Concettuale

1.1 Progettazione Concettuale con Class Diagrams

La progettazione concettuale è la fase in cui si modella il *cosa* deve essere rappresentato nella base di dati, ignorando i dettagli implementativi del DBMS. Sebbene la letteratura classica utilizzi il modello Entity-Relationship (ER), l'approccio moderno predilige l'uso dei **Class Diagrams UML**, di cui di seguito richiameremo i concetti fondamentali. Si assumerà che il lettore abbia già familiarità con i diagrammi UML in un contesto orientato agli oggetti, dunque ne faremo dei richiami piuttosto sommari e ci concentreremo sulle specificità della modellazione per basi di dati.

Il punto di partenza è un testo in linguaggio naturale (specifiche). La costruzione del diagramma segue una strategia di mapping semantico:

- **Sostantivi → Classi (Entità):** Rappresentano oggetti con esistenza autonoma (es. *Studiante, Corso*).
- **Verbi → Associazioni (Relazioni):** Descrivono i legami logici tra le classi (es. *Iscrive, Sostiene*).
- **Aggettivi o Specifiche → Attributi:** Proprietà atomiche che descrivono una classe (es. *Nome, Matricola*).

Gli elementi costitutivi di un Class Diagram sono:

Classe: Rappresentata da un rettangolo. Nella progettazione concettuale ci si concentra su *Nome* e *Attributi*. Le *Operazioni* (metodi) vengono solitamente omesse.

Associazione: Una linea che connette due classi. Indica che esiste una correlazione logica tra le istanze delle classi coinvolte.

Cardinalità: Indicano il numero minimo e massimo di istanze di una classe che possono essere legate a una singola istanza della classe opposta.

Tabella 1.1: Sintassi delle Cardinalità UML

Sintassi	Significato
1	Esattamente uno (Obbligatorio)
0..1	Zero o uno
* o 0..*	Zero o molti
1..*	Uno o molti (Almeno uno)

Un attributo si dice **totale** se ha cardinalità minima strettamente maggiore di zero, **parziale** altrimenti: quindi il primo deve sempre avere un valore, mentre per uno parziale il valore è opzionale.

Un attributo si dice **singolo** se ha cardinalità massima uguale a uno, **multiplo** altrimenti.

Un attributo è **atomico** se il database non ne percepisce la struttura interna. Gli attributi atomici possono essere:

- **Primitivi:** Rappresentano valori elementari non scomponibili.

- *Numerici*: `int`, `float`, `decimal`.
- *Alfanumerici*: `string`, `char`.
- *Logici*: `boolean`.
- **Predefiniti Strutturati**: Tipi di dato complessi forniti dal sistema e trattati come atomici dal DBMS.
 - *Temporal*: `date`, `time`, `timestamp`.
 - *Oggetti Binari*: `blob` (immagini, file), `clob` (testi estesi).

Un errore comune è modellare come attributo ciò che dovrebbe essere una classe. La regola empirica è:

1. Se l'informazione è un valore semplice (es. *Età*), è un **Attributo**.
2. Se l'informazione ha proprietà proprie o deve essere legata ad altre entità (es. *Indirizzo* inteso come via, civico e CAP), deve essere una **Classe** autonoma.



L'insieme degli attributi deve permettere di identificare univocamente ogni istanza della classe.

Osservazione 1.1. Sebbene in Java usi `-` (`private`) e `+` (`public`), nei diagrammi per database la visibilità è meno rilevante, ma mantenere il `-` per gli attributi aiuta a ricordare l'incapsulamento dei dati.

1.1.1 Ruoli nelle Associazioni

Un **ruolo** identifica la funzione specifica che un'istanza di una classe svolge all'interno di un'associazione. Viene indicato alle estremità della linea di associazione.

- **Utilità**: I ruoli sono essenziali per chiarire il significato del legame, specialmente quando l'associazione è *riflessiva* (connette una classe a sé stessa) o quando esistono più associazioni tra le stesse classi.
- **Naming Convention**: Solitamente si utilizza un sostantivo che descrive la responsabilità (es. *responsabile*, *supervisore*, *partecipante*).
- **Corrispondenza OOP**: Nella programmazione ad oggetti, il nome del ruolo coincide spesso con il nome dell'attributo (o del campo) che contiene il riferimento all'oggetto correlato.

Esempio 1.1 (Associazione Riflessiva o Ricorsiva). Si consideri la classe **Persona** e l'associazione *Genitorialità*. Senza ruoli, il legame sarebbe ambiguo. Assegnando i ruoli **genitore** e **figlio** alle due estremità, la semantica del modello diventa univoca.

Osservazione 1.2. Spesso si tende a omettere il nome dell'associazione (il verbo al centro della linea) se i ruoli sono già sufficientemente esplicativi. In molti schemi professionali vengono mostrati solo i ruoli, perché sono più vicini alla struttura logica finale del database.

1.1.2 Navigabilità e Direzionalità

Una differenza cruciale tra la modellazione OOP e quella per basi di dati riguarda la **navigabilità** delle associazioni.

- **In Java (OOP)**: Le associazioni sono spesso *unidirezionali* per favorire il disaccoppiamento tra le classi. La direzione indica quale oggetto mantiene il riferimento (puntatore) verso l'altro.

- **Nelle Basi di Dati:** Le associazioni sono intrinsecamente *bidirezionali*. Una relazione tra due entità rappresenta un fatto logico che può essere interrogato partendo da entrambi i lati (es. tramite operatori di *Join* in SQL).

Dunque nel class diagram concettuale per database, si evitano solitamente le frecce di navigabilità. Una linea semplice indica che l'informazione è disponibile per l'interrogazione in modo simmetrico. La "direzionalità" fisica verrà stabilita solo nella fase di progettazione logica attraverso il posizionamento delle chiavi esterne.

1.1.3 Tipi Enumerativi (Enumerazioni)

Quando il dominio di un attributo è costituito da un insieme di valori costante, discreto e predefinito, si utilizza il costrutto della **Enumerazione**. Si tratta di un tipo di dato speciale che permette a un attributo di assumere solo uno dei valori elencati in una lista chiusa.

In UML, le enumerazioni sono rappresentate come classi con lo stereotipo «enumeration». Non hanno attributi propri, ma contengono un elenco di valori letterali che rappresentano le possibili istanze del tipo enumerativo.



A differenza dei diagrammi in OOP, le enumerazioni **non sono classi**, ma semplicemente un tipo di dato speciale; **non hanno associazioni con le classi**.

1.1.4 Classi di Associazione

Nella modellazione concettuale, capita spesso che un'associazione tra due classi possieda delle proprietà proprie. Se un attributo non descrive intrinsecamente né la classe A né la classe B, ma esiste solo in funzione della loro relazione, ci troviamo di fronte alla necessità di una **Classe di Associazione**.

Si consideri, ad esempio, il caso classico di un sistema universitario: uno **Studente** sostiene un **Corso**. Vogliamo memorizzare il *voto* conseguito.

- Il *voto* non è un attributo dello **Studente** (egli ha voti diversi per corsi diversi).
- Il *voto* non è un attributo del **Corso** (il corso ha voti diversi per studenti diversi).

In un diagramma standard, saremmo costretti a omettere il voto o a inserirlo forzatamente in una delle due classi, creando un errore concettuale e ridondanza.

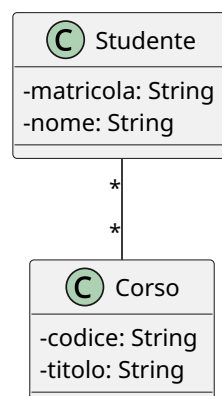


Figura 1.1: Associazione Studente-Corso.

L'UML risolve il problema permettendo a un'associazione di avere una propria classe collegata. Questa classe ha un doppio valore: è sia un legame logico tra le istanze, sia un contenitore di attributi.

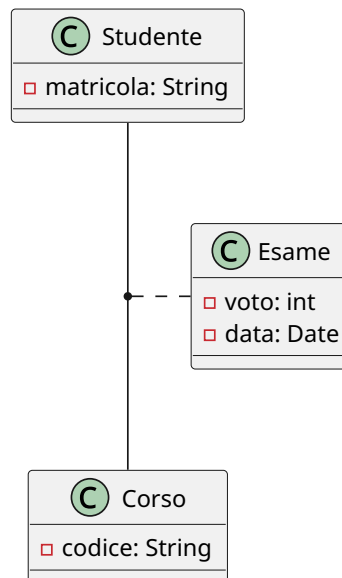


Figura 1.2: Studente-Corso con classe di associazione Esame.

Sintatticamente, si rappresenta con un rettangolo di classe collegato alla linea di associazione tramite una **linea tratteggiata**. Essa eredita implicitamente le "chiavi" di entrambe le classi coinvolte.

In molte fasi di progettazione logica, o per chiarezza di implementazione, la classe di associazione viene "esplicitata" (o reificata). La classe di associazione diventa una classe intermedia a tutti gli effetti, e l'associazione multi-a-molti originale si trasforma in due associazioni uno-a-molti.

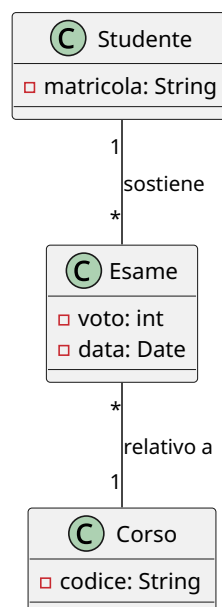


Figura 1.3: Reificazione della classe di associazione Esame.

L'esplicitazione è spesso necessaria perché molti strumenti di modellazione o linguaggi di programmazione non supportano nativamente il costrutto della classe di associazione. In questo caso, la classe intermedia (es. **Esame**) funge da "ponte" tra le due entità principali.

💡 Tip

La classe di associazione è significativa solo su associazioni *multi-a-multi*, ma occasionalmente si usa anche per altri tipi di associazioni.

1.1.5 Grado di un'Associazione e Relazioni Ternarie

Il **grado** di un'associazione è definito come il numero di classi coinvolte nel legame logico.

- **Associazioni Binarie (Grado 2):** Collegano due classi. Costituiscono la quasi totalità delle relazioni in un database.
- **Associazioni N-arie (Grado $N \geq 3$):** Collegano tre o più classi contemporaneamente. Il caso più comune è l'associazione **ternaria**.

Un'associazione ternaria si rende necessaria quando il legame logico esiste solo se si considerano tre entità simultaneamente; la scomposizione in relazioni binarie separate causerebbe una perdita irreparabile di informazione semantica. In UML si rappresenta graficamente interponendo un **rombo** tra le linee che collegano le tre classi.

Si consideri, ad esempio, uno scenario di logistica industriale in cui sono coinvolte tre classi: **Fornitore**, **Prodotto** e **Progetto** (cantiere di destinazione). L'atto del rifornimento è un evento atomico: il *Fornitore A* fornisce il *Prodotto X* specificamente per il *Progetto Y*. Se provassimo a mappare questo dominio con tre relazioni binarie (*Fornitore-Prodotto*, *Prodotto-Progetto*, *Fornitore-Progetto*), sapremmo quali prodotti vende un fornitore e quali prodotti usa un progetto, ma non sapremmo mai **quale specifico fornitore ha portato quel determinato prodotto a quel singolo progetto**.

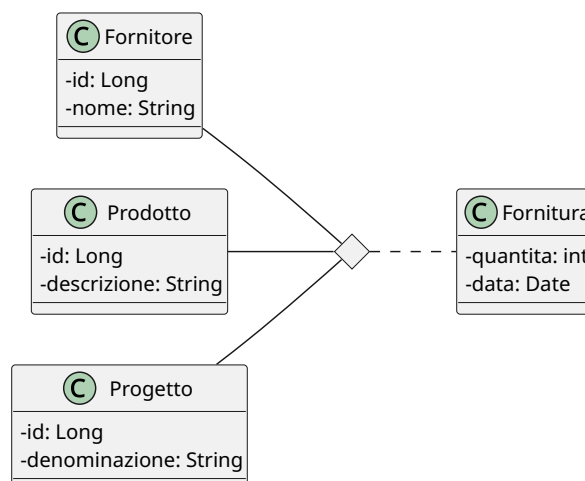


Figura 1.4: Esempio associazione ternaria con classe di associazione.

💡 Interpretazione di una associazione n-aria

Per comprendere a fondo l'associazione ternaria, si può pensare che una delle tre classi non si colleghi singolarmente alle altre due, ma alla loro **coppia**. Nel

🔦 nostro esempio, il **Progetto** non è legato separatamente a un **Prodotto** e a un **Fornitore**, ma è associato stabilmente alla coppia (*Prodotto*, *Fornitore*), intesa come specifica combinazione. Analogamente per le altre due classi.

Poiché i DBMS relazionali e i framework ORM (come JPA/Hibernate) non supportano le relazioni native a tre vie, si deve **esplicitare** (o reificare) l'associazione prima dell'implementazione.

La reificazione trasforma il rombo dell'associazione in una classe vera e propria, che chiameremo **AssegnazioneFornitura**. L'associazione ternaria originale viene così frantumata in tre associazioni binarie distinte di tipo **1-a-Molti**:

- Da **Fornitore** a **AssegnazioneFornitura** (Cardinalità $1 \rightarrow *$)
- Da **Prodotto** a **AssegnazioneFornitura** (Cardinalità $1 \rightarrow *$)
- Da **Progetto** a **AssegnazioneFornitura** (Cardinalità $1 \rightarrow *$)

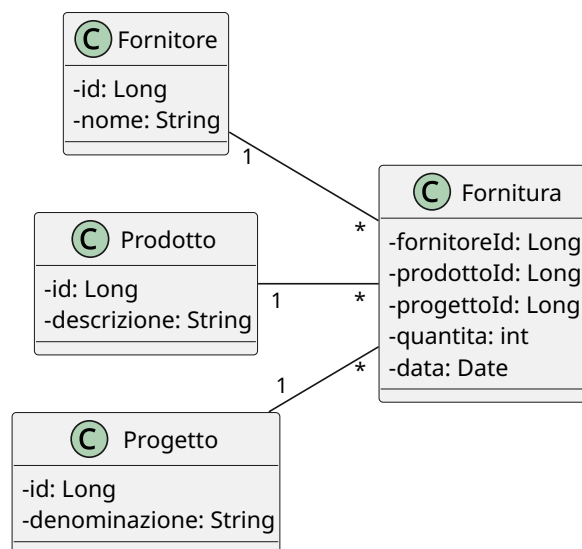


Figura 1.5: Reificazione dell'associazione ternaria.

Nella realtà, gli eventi n-ari catturano quasi sempre proprietà quantitative. Riprendendo l'esempio precedente, l'atto della fornitura genera intrinsecamente una *quantità* di pezzi consegnati e una *data* di consegna. Questi attributi non appartengono a nessuna delle tre classi isolate, ma alla loro combinazione. In UML si modella una **Classe di Associazione** (es. **Fornitura**) collegata tramite una linea tratteggiata al rombo dell'associazione ternaria.

Il processo di esplicitazione in questo scenario è immediato: la classe di associazione **Fornitura** viene promossa a classe centrale del cluster, inglobando gli attributi **quantità** e **data**. Le tre classi originarie si collegano a essa tramite relazioni binarie dirette.

⚠ La trappola delle cardinalità ternarie

Nelle associazioni ternarie UML, la cardinalità posta vicino a una classe (es. **Progetto**) indica quante istanze di quella classe possono combinarsi con una **coppia fissa** di istanze delle altre due classi (**Fornitore** e **Prodotto**). Questa logica è controintuitiva e differisce dai diagrammi ER classici (notazione di Chen), inducendo spesso i progettisti in errore. Sostituire subito la ternaria con una classe



esplicitata azzera ogni ambiguità interpretativa.

1.1.6 Aggregazione e Composizione

Nello sviluppo dei Class Diagrams, le associazioni standard indicano un legame logico simmetrico. Tuttavia, per modellare scenari in cui esiste un rapporto di possesso o di subordinazione strutturale, l'UML introduce le **relazioni tutto-parte** (part-whole relationships). Queste si dividono in due varianti fondamentali, distinte in base al vincolo sul ciclo di vita degli oggetti coinvolti: l'aggregazione e la composizione.

L'**aggregazione** rappresenta una relazione in cui un oggetto “Tutto” (aggregante) contiene o colleziona uno o più oggetti “Parte” (aggregati), ma questi ultimi mantengono un'esistenza autonoma all'interno del dominio. Si tratta di un legame “*has-a*” ad accoppiamento debole.

- **Sintassi UML:** Si rappresenta graficamente con una linea che termina con un **rombo vuoto** (bianco) posizionato sul lato della classe “Tutto”.
- **Ciclo di vita:** La distruzione dell'oggetto aggregante **non** comporta la distruzione degli oggetti aggregati.

Esempio 1.2 (Università e Professore). Si consideri la classe **Università** (il Tutto) e la classe **Professore** (la Parte). Un'università raccoglie un insieme di docenti. Tuttavia, se l'università dovesse cessare la sua attività, i singoli professori non verrebbero eliminati dal sistema: essi continuerebbero ad esistere come entità autonome e potrebbero essere associati ad altri atenei.

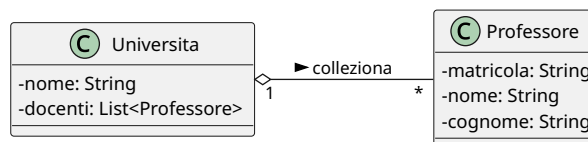


Figura 1.6: Esempio di aggregazione

La **composizione** è una forma di aggregazione più stringente, esclusiva e gerarchica. In questo scenario, l'oggetto “Parte” è una componente indissolubile del “Tutto”. Una parte può appartenere a un solo tutto alla volta e non ha alcuna autonomia esistenziale.

- **Sintassi UML:** Si rappresenta con una linea che termina con un **rombo pieno** (nero) posizionato sul lato della classe “Tutto”.
- **Ciclo di vita:** Il ciclo di vita delle parti è totalmente subordinato a quello del tutto. Di conseguenza, la distruzione dell'oggetto composto (Tutto) determina l'istanza di **distruzione automatica** di tutte le sue parti. La cardinalità dal lato del tutto è rigorosamente 1 o 0..1.

Esempio 1.3 (Ordine ed Elemento d'Ordine). Si consideri un sistema di e-commerce con le classi **Ordine** (il Tutto) e **RigaOrdine** (la Parte, che descrive il prodotto e la quantità). Una riga d'ordine ha significato logico solo se contestualizzata all'interno dello specifico acquisto. Se l'utente decide di eliminare l'intero **Ordine**, il sistema deve rimuovere a cascata tutte le relative istanze di **RigaOrdine**.

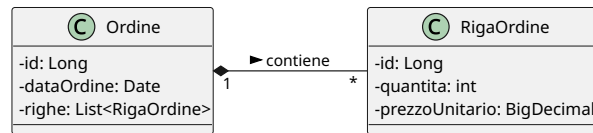


Figura 1.7: Esempio di composizione

🔍 L'aggregazione non ha alcuna necessità tecnica stringente: una semplice associazione fa esattamente lo stesso identico lavoro. A livello di tabelle, sia un'associazione generica sia un'aggregazione si tradurranno nello stesso modo, la differenza è puramente semantica e documentale, non implementativa. Alcuni autorevoli ingegneri del software (tra cui Martin Fowler, uno dei padri dell'architettura software moderna) sostengono da anni che l'aggregazione UML sia fondamentalmente inutile perché la linea di demarcazione con l'associazione standard è troppo sfumata e crea più ambiguità che benefici. Fowler la definisce ironicamente "un effetto placebo grafico".

Il caso della composizione è invece più significativo, perché il diagramma fornisce un'informazione in più: la parte non può esistere senza il tutto.

1.1.7 Generalizzazioni/Specializzazioni

L'ereditarietà modella una relazione di tipo “IS-A” (è-un) tra una classe più generale, detta **Generalizzazione** (o Superclasse, per analogia con i contesti OOP), e una o più classi più specifiche, dette **Specializzazioni** (o Sottoclassi).

Le specializzazioni ereditano automaticamente tutti gli attributi, i ruoli e le associazioni della generalizzazione, senza necessità di ridefinirli, e possono estendere il modello introducendo attributi o associazioni propri.

Sintassi UML: Si rappresenta graficamente con una linea che termina con una **freccia con la punta triangolare vuota** rivolta verso la generalizzazione.

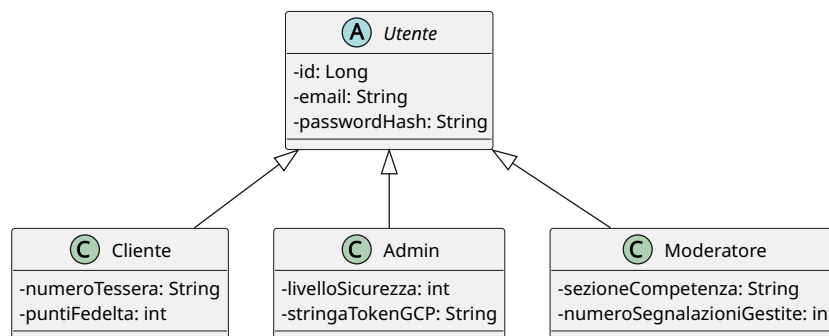


Figura 1.8: Esempio di ereditarietà

È fondamentale comprendere che **l'ereditarietà non esiste nel modello relazionale delle basi di dati** (o almeno, non secondo il modello relazionale dei dati che noi studieremo). I database relazionali non possiedono strutture gerarchiche native; essi conoscono esclusivamente il concetto di tabella (relazione), che è per sua natura una struttura “piatta” e bidimensionale.

La generalizzazione appartiene quindi in modo esclusivo al livello della **progettazione concettuale**. In questa fase, lo scopo del Class Diagram è modellare il dominio reale nel modo più fedele e naturale possibile, catturandone la semantica e i vincoli logici

(ovvero i rapporti di sotto-tipo/sopra-tipo) ed eliminando qualsiasi vincolo tecnologico. Il problema di come “appiattare” questa gerarchia per adattarla a un database relazionale sarà l’obiettivo centrale della successiva fase di progettazione logica.

Una generalizzazione può essere classificata combinando due tipi di vincoli ortogonali, utili a descrivere la natura delle istanze nel dominio reale:

1. **Completezza (Totale vs Parziale):**

- **Totale (Complete):** Ogni istanza della superclasse deve obbligatoriamente appartenere ad almeno una delle subclassi. Non esistono elementi “generici” (corrisponde al concetto di *Classe Astratta* in OOP).
- **Parziale (Incomplete):** Un’istanza della superclasse può esistere senza dover per forza appartenere a una delle subclassi catalogate.

2. **Disgiunzione (Disgiunta vs Sovrapposta):**

- **Disgiunta (Disjoint):** Un’istanza della superclasse può appartenere al massimo a **una sola** delle subclassi. Le sottoclassi sono mutuamente esclusive.
- **Sovrapposta (Overlapping):** Una singola istanza della superclasse può appartenere contemporaneamente a più subclassi distinte.

Ci sono dunque quattro combinazioni possibili. Nel diagramma UML, i vincoli della generalizzazione si indicano inserendo un’etichetta testuale racchiusa tra parentesi graffe (*Generalization Set*) in prossimità delle linee di ereditarietà, specificando la coppia di proprietà (es: {complete, overlapping}).

Esempio 1.4 (Esempi di Vincoli). Si considerino i seguenti scenari aziendali:

- **Totale / Disgiunta:** La superclasse *ContoBancario* ha come subclassi *ContoCorrente* e *ContoDeposito*. Un conto deve essere obbligatoriamente di un tipo o dell’altro, e non può essere entrambe le cose contemporaneamente.
- **Parziale / Sovrapposta:** La superclasse *Persona* ha come subclassi *Studente* e *Dipendente*. All’interno dell’anagrafica universitaria, una persona può essere solo un cittadino esterno (parziale), oppure può essere contemporaneamente uno studente che lavora per l’ateneo (sovrapposta).

1.2 La Documentazione di Supporto: I Dizionari dei Dati

Il Class Diagram, pur essendo uno strumento visivo formidabile, non è espressivamente sufficiente a catturare l’intera semantica di un dominio applicativo. Elementi come il significato preciso di un attributo, il contesto d’uso di un’associazione o le regole aziendali complesse (*Business Rules*) non possono essere espressi tramite nodi e archi.

La progettazione concettuale si completa quindi attraverso i **Dizionari dei Dati**, estensioni testuali strutturate che definiscono formalmente ogni componente dello schema concettuale. Questi documenti rappresentano la specifica dei requisiti su cui si baserà lo sviluppo dello strato di persistenza e della logica di business.

I dizionari dei dati sono:

- **Dizionario delle Classi:** descrive in modo univoco ogni entità del sistema. Si presenta come una tabella in cui ogni riga rappresenta una classe. Per ogni classe vengono forniti il nome (prima colonna), una descrizione in linguaggio naturale per evitare ambiguità terminologiche (seconda colonna), nome, tipo e descrizione di ciascun attributo (terza colonna).
- **Dizionario delle Associazioni:** si presenta come una tabella analoga alla precedente, in cui ogni riga rappresenta un’associazione. Per ogni associazione vengono

forniti il nome (prima colonna), descrizione (seconda colonna), le classi coinvolte con ruolo, molteplicità e descrizione (terza colonna).

- **Dizionario dei Vincoli:** è il documento più critico. I **Vincoli di Integrità Esogeni** (o regole di business) sono restrizioni sul dominio dei dati che **non possono** essere espresse graficamente nel Class Diagram tramite la normale sintassi UML. Mentre una cardinalità $1..*$ esprime un vincolo strutturale grafico, una regola del tipo “*Un cliente non può effettuare un ordine se il suo account è sospeso*” non ha un simbolo UML dedicato. Viene quindi codificata nel dizionario dei vincoli in linguaggio naturale o tramite espressioni logico-matematiche formali (come l’OCL - *Object Constraint Language*). Si presenta come una tabella a due colonne in cui ogni riga rappresenta un vincolo, nella prima colonna si inserisce il nome del vincolo oppure un suo identificatore univoco (es. BR - Business Rule), nella seconda colonna la sua descrizione in linguaggio naturale.

Esempio 1.5 (Catalogo dei Vincoli d’Integrità).

BR01 (Coerenza Temporale): La data di spedizione di un `Ordine` deve essere strettamente successiva o uguale alla sua data di creazione.

BR02 (Integrità Finanziaria): L’attributo `totale` di un `Ordine` deve essere matematicamente pari alla sommatoria del prodotto tra `quantita` e `prezzoUnitario` di tutte le istanze di `RigaOrdine` a esso collegate.

BR03 (Regola di Business): Un `Utente` con ruolo `Admin` non può associare a sé stesso un’istanza di `Ordine` in veste di `acquirente`.



Nel dizionario dei vincoli non vanno inseriti vincoli che sono già espressi nel class diagram.

I vincoli devono garantire la qualità interna dei dati nella base di dati.

Modello Relazionale e Progettazione Logica

2.1 Introduzione

Il passaggio dalla progettazione concettuale alla progettazione logica rappresenta la transizione dal *cosa* deve essere rappresentato al *come* organizzare concretamente i dati all'interno di un sistema di gestione di basi di dati (DBMS). Questo processo richiede l'adozione di uno specifico **modello dei dati**.

Sebbene la storia dell'informatica abbia visto l'evoluzione di diversi paradigmi — come il modello gerarchico, quello reticolare e quello orientato agli oggetti —, la nostra trattazione si concentrerà sul **modello relazionale**, che costituisce lo standard industriale prevalente. Vale la pena notare che i DBMS moderni più diffusi (come PostgreSQL e Oracle) adottano un'evoluzione di questo approccio, denominata **modello relazionale a oggetti**, la quale estende le fondamenta del modello relazionale puro integrando costrutti tipici del paradigma OOP.

Uno dei principali vantaggi nell'aver adottato i **Class Diagrams UML** nella fase concettuale risiede nella loro assoluta indipendenza tecnologica: essi si adattano teoricamente a qualunque modello logico finale. Di contro, questa spiccata espressività introduce un inevitabile **disallineamento semantico** (*impedance mismatch*) quando l'obiettivo è la traduzione in uno schema relazionale puro.

Nel passaggio alla progettazione logica relazionale, le principali sfide strutturali da affrontare saranno dettate dai rigidi vincoli di atomicità e piattezza del dato tipici delle tabelle:

- **Attributi strutturati:** Il modello relazionale esige che i valori degli attributi siano atomici e non strutturati. Non è quindi possibile nidificare record, oggetti o tipi di dato complessi all'interno di una colonna.
- **Attributi a valore multiplo (multivalore):** Ogni cella di una tabella relazionale può contenere esclusivamente un singolo valore atomico, vietando l'uso nativo di collezioni, array, liste o set all'interno di un singolo attributo.
- **Assenza di ereditarietà:** Il modello relazionale puro non supporta nativamente le gerarchie di generalizzazione-specializzazione, costringendo il progettista ad applicare strategie di “appiattimento” o di scomposizione strutturale.

2.2 Concetti Fondamentali

Il **modello relazionale** si basa sul concetto matematico di relazione. Questo modello rappresenta la struttura logica e piatta su cui verranno mappati i dati dell'applicazione. Per formalizzare rigorosamente il modello, è necessario definire i suoi tre componenti atomici: i domini, lo schema e la relazione stessa.

Un **dominio** (indicato con D) è un insieme di valori atomici, ovvero non ulteriormente scomponibili. Ogni dominio è caratterizzato da un nome e da un tipo di dato (es: `int`, `varchar`, `date`).

Osservazione 2.1 (Il Tipo di Dato come Dominio Implicito). Nella transizione dalla teoria all'ingegneria del software, il **tipo di dato** (es. `int`, `varchar`, `date`) definisce l'insieme universo di partenza, agendo come **dominio massimo implicito** dell'attributo.

Eventuali restrizioni logiche (come un *range* di valori validi per un voto universitario o un'età) si configurano come un **sottoinsieme** di tale dominio base. Vedremo in seguito come vengono implementate limitazioni specifiche.

Uno **schema relazionale** (o schema di tabella) definisce la struttura logica dei dati. Esso è costituito da un nome R e da un insieme di attributi $A_1, A_2, \dots, A_n$, a ciascuno dei quali è associato un dominio D_i , che denoteremo anche con $dom(A_i)$.

Lo schema viene denotato formalmente come:

$$R(A_1 : D_1, A_2 : D_2, \dots, A_n : D_n)$$

Lo schema relazionale corrisponde dunque esattamente al concetto di **Classe**. Esso definisce la struttura dei dati, i nomi dei campi e i rispettivi tipi, agendo come un “blueprint” statico.

Matematicamente, date le premesse precedenti, definiamo il dominio totale dello schema come il prodotto cartesiano dei domini dei singoli attributi:

$$D_{tot} = D_1 \times D_2 \times \dots \times D_n$$

Una **relazione** r sullo schema R è un sottoinsieme finito del prodotto cartesiano dei domini degli attributi:

$$r \subseteq D_1 \times D_2 \times \dots \times D_n$$

Gli elementi che compongono questo sottoinsieme sono detti **tuple** (o *n*-uple). Una *tuple* è un vettore di valori $(v_1, v_2, \dots, v_n)$ tale che ogni $v_i \in D_i$.



La corrispondenza con OOP è la seguente:

- Una **Tupla** corrisponde al singolo **Oggetto** (una istanza della classe contenente dati reali).
- La **Relazione** corrisponde all'insieme di tutti gli oggetti di quella classe **attualmente persistiti** nel database in un determinato istante temporale (lo stato corrente della tabella).

Esempio 2.1 (Formalizzazione di un'Anagrafica). Definiamo lo schema relazionale per la classe **Studente**:

$$STUDENTE(\text{matricola} : \text{String}, \text{nome} : \text{String}, \text{età} : \text{Integer})$$

Il prodotto cartesiano di questi tre domini genererebbe infinite combinazioni (comprese matricole inesistenti accoppiate a nomi casuali ed età assurde).

La **relazione** $r(STUDENTE)$ conterrà invece solo le tuple degli studenti reali inseriti nel sistema:

$$r = \{('654321', 'Giovanni', 22), ('123456', 'Mario', 24)\}$$

Osservazione 2.2. È significativo evidenziare che una relazione è *un insieme*, dunque non ammette duplicati e non rappresenta una struttura ordinata.

In sintesi:

Nel passaggio dalla progettazione concettuale alla modellazione logica, i concetti del paradigma orientato agli oggetti (*OOP*) e del Class Diagram si traducono come segue:

- **Classi** → **Schemi Relazionali**: Le classi si traducono in schemi di relazione, identificati da un nome (corrispondente al nome della classe) e da un insieme di attributi.
- **Attributi** → **Domini**: A ciascun attributo dello schema è associato un **dominio**, ovvero l'insieme di tutti i possibili valori atomici che quell'attributo può assumere (equivalente al tipo di dato in programmazione).
- **Istanze** → **Relazione (Stato)**: Una **relazione** è un sottoinsieme discreto e finito del dominio totale (il prodotto cartesiano di tutti i singoli domini). Essa rappresenta l'insieme delle tuple (istanze o oggetti) effettivamente memorizzate e persistite nel sistema in un dato momento temporale.

2.3 Le Chiavi

Fino ad ora abbiamo analizzato come tradurre classi e attributi del Class Diagram nel modello relazionale, lasciando in sospeso la traduzione delle associazioni. Per poter modellare le relazioni tra tabelle, è prima necessario introdurre il concetto di chiave.

Sia $R(A_1, A_2, \dots, A_n)$ uno schema di relazione. Un sottoinsieme di attributi $SK \subseteq \{A_1, A_2, \dots, A_n\}$ si definisce **superchiave** se i suoi valori identificano in modo univoco ogni singola tupla all'interno di qualsiasi istanza valida della relazione.

Esempio 2.2 (Superchiavi di un'Anagrafica). Si consideri lo schema:

STUDENTE(CF, matricola, nome, cognome, dataNascita)

I singoletti $\{CF\}$ e $\{matricola\}$ sono superchiavi. Di conseguenza, **qualsiasi sottoinsieme che li contenga** (es: $\{CF, nome\}$ o $\{matricola, cognome\}$) è a sua volta una superchiave.

Osservazione 2.3 (Ipotesi di Esistenza). Nella fase di progettazione concettuale si assume l'ipotesi che ogni istanza di una classe sia distinguibile dalle altre. Questa proprietà ci garantisce che **esiste sempre almeno una superchiave** per ogni schema relazionale: nella peggiore delle ipotesi (assenza di identificatori naturali), la superchiave coinciderà con l'insieme di tutti gli attributi dello schema.

Un sottoinsieme di attributi K è una **chiave candidata** se è una **superchiave minimale**. Dire che una superchiave è *minimale* significa che è **irriducibile**: eliminando un qualsiasi attributo da K , l'insieme rimanente perde la proprietà di univocità e cessa di essere una superchiave.

Questo **non** significa che debba avere il numero minimo di attributi in assoluto (cardinalità minima). Se nello schema coesistono una superchiave irriducibile di cardinalità 1 e una di cardinalità 2, sono entrambe, con pari dignità, chiavi candidate.

È evidente che una superchiave composta da un unico attributo (un singoletto) è di fatto e necessariamente una chiave candidata.

Esempio 2.3 (Chiave Candidata Composta). Si consideri lo schema che mappa la classe degli esami universitari:

ESAME(matricola, corso, voto, data)

In questo scenario, il sottoinsieme {matricola, corso} rappresenta una chiave candidata (di fatto, è l'unica chiave candidata in questo caso). Essa è minimale perché se eliminassimo **corso**, la sola **matricola** non garantirebbe l'univocità (uno studente può sostenere più esami); viceversa, eliminando **matricola**, il solo **corso** conterrebbe i voti di tutti gli studenti.

Poiché in uno schema relazionale possono coesistere più chiavi candidate, il progettista deve sceglierne una da eleggere come identificatore ufficiale delle tuple. La chiave candidata scelta prende il nome di **Chiave Primaria** (*Primary Key* - PK).

La selezione della chiave primaria è una decisione puramente ingegneristica e architeturale. Nello schema relazionale scritto, la chiave primaria si indica **sottolineando** gli attributi che la compongono.

Esempio 2.4 (Scelta della PK). Riprendendo lo schema **STUDENTE**, le chiavi candidate sono CF e **matricola**. Sceglieremo come chiave primaria la **matricola**:

STUDENTE(CF, matricola, nome, cognome)

La **Chiave Esterna** (*Foreign Key* - FK) è lo strumento fondamentale del modello relazionale per tradurre le associazioni.

Siano R_1 e R_2 due schemi relazionali (non necessariamente distinti). Un insieme di attributi FK di R_1 è una chiave esterna che punta a R_2 se:

1. Gli attributi in FK sono in corrispondenza biunivoca con gli attributi che compongono la **chiave primaria** (PK) di R_2 .
2. I domini degli attributi corrispondenti sono identici ($dom(FK_i) = dom(PK_i)$).

Il vincolo impone che per ogni tupla t_1 nell'istanza corrente di R_1 , il valore di $t_1[FK]$ deve essere nullo (NULL), oppure coincidere esattamente con il valore $t_2[PK]$ di **una tupla** t_2 **attualmente esistente** nell'istanza di R_2 .

Nello schema testuale, la presenza di una chiave esterna si denota convenzionalmente con una **doppia sottolineatura**.

2.4 Traduzione delle Associazioni

2.4.1 Associazioni 1 a 1

Un'associazione **1 a 1** tra due classi A e B indica che a un'istanza di A corrisponde al massimo un'istanza di B , e viceversa. Nel modello relazionale, questo legame si traduce introducendo una **Chiave Esterna (FK)** in uno dei due schemi che punta alla Chiave Primaria (PK) dell'altro.

Esempio 2.5 (Associazione esattamente 1 a 1). Si consideri un'associazione in cui la partecipazione è rigida e obbligatoria per entrambe le classi ($1..1 \leftrightarrow 1..1$), ad esempio tra l'entità **STUDENTE**(matricola, nome, cognome) e il suo **FASCICOLO**(codice, dataCreazione). Ogni studente possiede esattamente un fascicolo e ogni fascicolo appartiene esattamente a uno studente. Traduciamo il modello aggiungendo la chiave esterna nello schema del fascicolo:

- STUDENTE(matricola, nome, cognome)
- FASCICOLO(codice, dataCreazione, matricolaStudente)

Affinché la relazione rimanga rigorosamente 1 a 1, non è sufficiente definire la chiave esterna. Se ci limitassimo a questo, la tabella che ospita la FK potrebbe avere più tuple che puntano alla stessa PK della tabella bersaglio, trasformando di fatto la relazione in una 1 a N.

Riprendendo l'esempio precedente, se l'attributo **studente** nello schema **FASCICOLO** fosse definito unicamente come chiave esterna, nulla impedirebbe al database di registrare due tuple distinte di fascicoli (es. codice 'F01' e 'F02') aventi entrambe il valore **studente** = '12345'. Questo corromperebbe la semantica concettuale, asserendo che la matricola 12345 possiede *molti* fascicoli.

Per garantire la biunivocità dell'associazione, è obbligatorio imporre un **vincolo di univocità** sugli attributi che compongono la chiave esterna. Matematicamente, se lo schema R_1 contiene la chiave esterna FK verso R_2 , deve valere che per ogni coppia di tuple distinte $t_x, t_y \in r(R_1)$:

$$t_x[FK] \neq t_y[FK] \quad (\text{escludendo i valori NULL})$$

Q Perché usare una sola Chiave Esterna?

Vista la forte simmetria di un'associazione 1 a 1, potrebbe sorgere spontaneo il dubbio: perché non posizionare una chiave esterna in *entrambi* gli schemi (es. aggiungere **fascicolo** dentro **STUDENTE** e **studente** dentro **FASCICOLO**)? La teoria relazionale esclude questa pratica per tre ragioni fondamentali:

1. **Ridondanza e Rischio di Anomalie:** Il legame logico verrebbe salvato due volte. Se per errore un sistema aggiornasse un lato senza allineare l'altro (es. lo **Studente** punta al **Fascicolo A**, ma il **Fascicolo A** punta a un altro **Studente**), il database entrerebbe in uno stato incoerente.
2. **Bidirezionalità Intrinseca:** Come già stabilito, la presenza di una singola chiave esterna è matematicamente sufficiente per stabilire un'uguaglianza logica tra le tuple. Il motore del database può navigare il legame in entrambe le direzioni senza bisogno di puntatori doppi.
3. **Dipendenza Circolare (Deadlock di Inserimento):** Poiché in questo esempio la partecipazione è obbligatoria da ambo i lati, le rispettive chiavi esterne dovrebbero avere un vincolo NOT NULL. Tuttavia, questo creerebbe un paradosso logico: non si potrebbe inserire uno studente nel database perché manca il suo fascicolo, e non si potrebbe inserire il fascicolo perché manca lo studente a cui associarlo. Inserire la chiave esterna in un solo schema spezza questo circolo vizioso.

Osservazione 2.4. In alcuni casi estremi di forte coesione, le due tabelle possono persino essere fuse in un unico schema relazionale. Approfondiremo questo aspetto nel prossimo paragrafo.

Nell'esempio precedente si aveva una situazione di relazione esattamente 1 a 1 e la scelta dello schema in cui inserire la chiave esterna era indifferente. In altri casi, la scelta di quale delle due tabelle debba ospitare la FK dipende dalle **cardinalità minime** (l'opzionalità o obbligatorietà dell'associazione): se la classe A partecipa in modo opzionale (0..1) e la classe B in modo obbligatorio (1..1), la chiave esterna **deve essere inserita nello schema B**. In questo modo, la FK in B potrà essere dichiarata NOT NULL (vincolo di esistenza garantito). Se la mettessimo in A , saremmo costretti ad ammettere valori NULL per le istanze di A che non partecipano all'associazione, sprecando spazio e riducendo l'eleganza relazionale.

Esempio 2.6 (Direzione di un Dipartimento). Si consideri l'associazione 1 a 1 tra `Professore` (0..1) e `Dipartimento` (1..1): un professore può dirigere al massimo un dipartimento, ma un dipartimento deve avere esattamente un direttore.

Seguendo la regola, posizioniamo la chiave esterna nel `Dipartimento`:

- `PROFESSORE`(codiceDocente, nome, cognome)
- `DIPARTIMENTO`(codice, nome, codiceDirettore)

Vincoli su `DIPARTIMENTO.codiceDirettore`:

- **Foreign Key:** punta a `PROFESSORE.codiceDocente`
- **UNIQUE:** per garantire che lo stesso professore non diriga due dipartimenti (vincolo 1:1).
- **NOT NULL:** per garantire che ogni dipartimento abbia un direttore (cardinalità minima 1).

Q Controllo Applicativo vs Vincolo Strutturale (UNIQUE)

In alcuni contesti didattici o in framework di sviluppo specifici, è possibile omettere la dichiarazione esplicita del vincolo **UNIQUE** per le associazioni 1 a 1.

In questi scenari, si assume un'ipotesi di **coerenza applicativa**: si dà per scontato che sia la logica del software sovrastante (es. i controlli nel codice backend) a impedire l'inserimento di tuple multiple, in accordo con la natura del dominio ("non può mai capitare che l'entità sia associata a molti").

Tuttavia, dal punto di vista della solidità ingegneristica e dell'integrità fisica dei dati (*defensive programming*), l'omissione del vincolo **UNIQUE** rende il database strutturalmente identico a un'associazione 1 a Molti. Fissare il vincolo a livello di DBMS rimane la pratica raccomandata per garantire che i dati non vengano mai corrotti, indipendentemente da eventuali bug del software applicativo.

2.4.2 Associazioni 1 a Molti (1:N)

Nel modello relazionale, questa associazione si traduce secondo una regola fissa e inderogabile, dettata dal vincolo di atomicità degli attributi (assenza di attributi multivalore): la **Chiave Esterna (FK) deve essere sempre inserita nello schema dal lato "Molti" (B)**, e punterà alla Chiave Primaria (PK) dello schema dal lato "Uno" (A).

A differenza dell'associazione 1 a 1, in questo caso **non** si deve applicare alcun vincolo di univocità (**UNIQUE**) sulla chiave esterna. È infatti intrinseco nella natura dell'associazione 1:N che più tuple della relazione *B* possano possedere lo stesso valore per l'attributo FK, puntando così alla medesima istanza di *A*.

Esempio 2.7 (Impiegati in un Dipartimento). Si consideri l'associazione tra `Dipartimento` (1) e `Impiegato` (N). Un dipartimento ospita molti impiegati, ma un impiegato afferisce a un solo dipartimento. La chiave esterna viene posizionata nello schema `IMPIEGATO`:

- `DIPARTIMENTO`(codice, nome, sede)
- `IMPIEGATO`(matricola, nome, cognome, codiceDipartimento)

Osservazione 2.5 (Monodirezionalità Apparente). Anche in questo caso, la posizione fisica della chiave esterna genera una *monodirezionalità apparente*. Guardando lo schema, sembra che un impiegato "conosca" il proprio dipartimento, ma che il dipartimento sia ignaro dei propri impiegati (poiché non ha attributi che li referenziano).

Come già chiarito, questa è solo un'illusione strutturale: l'informazione è conservata intatta dalla logica dei valori. Il database è sempre in grado di ricavare l'elenco degli

impiegati di uno specifico dipartimento, semplicemente interrogando la tabella **IMPIEGATO** e filtrando le tuple che possiedono quel determinato `codiceDipartimento`.

2.4.3 Associazioni Molti a Molti (N:N)

Nei casi precedenti (1:1 e 1:N), la traduzione avveniva inserendo una chiave esterna direttamente in uno dei due schemi coinvolti. Se provassimo ad applicare questa logica a un'associazione N:N, violeremmo il vincolo fondamentale del modello relazionale:

- Inserire la chiave esterna di **CORSO** nello schema **STUDENTE** costringerebbe una tupla studente a contenere un attributo **multivalore** (una lista di codici di corsi).
- Viceversa, inserire la chiave esterna di **STUDENTE** in **CORSO** creerebbe una lista di matricole dentro la tupla del corso.

Poiché il modello relazionale ammette esclusivamente valori atomici, l'uso di chiavi esterne dirette è matematicamente impossibile.

Per tradurre un'associazione Molti a Molti, la teoria relazionale impone la creazione di un **terzo schema di relazione autonomo**, comunemente chiamato *tabella di giunzione*, *tabella ponte* o *relazione associativa*.

Questa nuova tabella conterrà esclusivamente (almeno nella sua forma base) le chiavi esterne che puntano alle chiavi primarie dei due schemi originari. La **Chiave Primaria** di questa nuova tabella sarà composta dall'unione delle due chiavi esterne (chiave primaria composta).

Esempio 2.8 (Studenti e Corsi). Si consideri l'associazione N:N tra **STUDENTE** e **CORSO**. Creiamo la tabella ponte **FREQUENZA**:

- **STUDENTE**(matricola, nome)
- **CORSO**(codice, titolo, cfu)
- **FREQUENZA**(matricolaStudente, codiceCorso)

Vincoli sulla tabella **FREQUENZA**:

- **Primary Key:** È composta dalla coppia (matricolaStudente, codiceCorso). Questo garantisce che uno studente non possa essere iscritto due volte allo stesso identico corso.
- **Foreign Key 1:** matricolaStudente punta a **STUDENTE.matricola**.
- **Foreign Key 2:** codiceCorso punta a **CORSO.codice**.

Osservazione 2.6 (Delegazione della Complessità). Dal punto di vista della progettazione (struttura statica), il pattern della tabella di giunzione rende la risoluzione delle associazioni N:N un processo banale e standardizzato. La reale complessità viene delegata alla fase operativa (interrogazione dei dati). Per estrarre, ad esempio, i nomi degli studenti e i titoli dei corsi da loro frequentati, il DBMS dovrà eseguire computazionalmente onerosi **JOIN multipli**, attraversando la tabella ponte per ricongiungere i dati disaccoppiati.

È chiaro che questo metodo per rendere le associazioni è del tutto generale e, in teoria, potrebbe essere usato per tutti i tipi di associazione; tuttavia, si tratta ovviamente di una cattiva pratica in virtù del costo computazionale che è stato appena discusso.

Classi di Associazione

Nel modellamento concettuale tramite Class Diagram UML, capita frequentemente che un'associazione molti a molti possieda a sua volta degli attributi (formalizzati tramite una

Classe di Associazione). Un esempio tipico è la relazione di superamento di un esame tra uno studente e un corso, la quale porta con sé attributi come il *voto*, la *data* e la *lode*.

La traduzione di questo costrutto nel modello relazionale segue fedelmente la regola dello schema ponte vista in precedenza. Gli attributi propri dell'associazione vengono semplicemente inseriti come **colonne aggiuntive all'interno della tabella di giunzione**.

Esempio 2.9 (Mappatura della Classe di Associazione Esame). Estendiamo l'esempio precedente trasformando lo schema ponte FREQUENZA in uno schema ESAME che ospiti anche i dati del superamento del corso:

- STUDENTE(matricola, nome, cognome)
- CORSO(codice, titolo, cfu)
- ESAME(matricolaStudente, codiceCorso, voto, data, lode)

Nello schema ESAME, la chiave primaria rimane la coppia composta (matricolaStudente, codiceCorso), il che modella il vincolo per cui uno studente può registrare al massimo un voto per un determinato corso. Gli attributi *voto*, *data* e *lode* diventano attributi non chiave della tabella di giunzione.

Associazioni n-arie

La logica della tabella di giunzione, introdotta per le associazioni binarie Molti a Molti, scala in modo naturale e identico per le **associazioni n-arie** (con $n > 2$).

Quando nel diagramma concettuale un'associazione coinvolge tre o più classi (ad esempio un'associazione ternaria), la traduzione nel modello relazionale richiede la creazione di un unico **schema ponte** dedicato. Questo schema assolverà il compito di legare tutte le entità simultaneamente e conterrà:

- Una **Chiave Esterna (FK)** per ciascuna delle n entità partecipanti, che punta alla rispettiva chiave primaria.
- Una **Chiave Primaria (PK) composta** dall'unione di tutte le chiavi esterne coinvolte (salvo casi particolari in cui vincoli di cardinalità massima pari a 1 su specifici rami consentano di escludere alcune FK dalla chiave primaria).
- Gli eventuali attributi dell'associazione (o della Classe di Associazione), inseriti semplicemente come colonne non chiave.

Esempio 2.10 (Associazione Ternaria di Fornitura). Si consideri un'associazione ternaria che modella la fornitura di materiali aziendali: un determinato **Fornitore** vende uno specifico **Prodotto** destinato a un particolare **Progetto**, in una certa **quantità**.

La traduzione genera tre schemi base e un quarto schema ponte per l'associazione ternaria:

- FORNITORE(partitaIva, nomeAzienda)
- PRODOTTO(codiceProdotto, descrizione)
- PROGETTO(idProgetto, budget)
- FORNITURA(partitaIva, codiceProdotto, idProgetto, quantità)

Nello schema FORNITURA, la chiave primaria è la tripla di attributi (partitaIva, codiceProdotto, idProgetto). L'attributo *quantità* è un attributo dell'associazione che dipende funzionalmente dall'intera tripla, in quanto esprime quanti pezzi di quel prodotto specifico sono stati comprati da quel fornitore per quel preciso progetto.

2.5 Ristrutturazione dello Schema Concettuale

Il Class Diagram realizzato durante la progettazione concettuale ha l'unico scopo di modellare la semantica del dominio applicativo, ignorando i limiti tecnologici. Tuttavia,

avvicinandosi alla progettazione logica, l'adozione dello specifico modello relazionale fa emergere vincoli strutturali e di performance che rendono il diagramma originario inefficiente o inapplicabile.

Prima di procedere alla traduzione in tabelle, è quindi necessaria una fase di **Ristrutturazione del Class Diagram**, durante la quale il modello viene riadattato e "sporcato" con dettagli implementativi. Questa fase si articola in diverse analisi sequenziali.

2.5.1 Analisi delle Chiavi: Chiavi Naturali vs Surrogate

La prima operazione consiste nel verificare gli identificatori (chiavi primarie) scelti per le classi. Nel modello concettuale, è prassi utilizzare come identificatori degli attributi già esistenti nel dominio della realtà, noti come **Chiavi Naturali**. Tuttavia, capita spesso che per identificare univocamente un'istanza nel mondo reale sia necessario unire molti attributi, creando una **chiave primaria composta** molto ingombrante.

L'uso di chiavi primarie formate da tre, quattro o più attributi genera due enormi problemi nel database relazionale:

1. **Propagazione delle Foreign Key:** Poiché le associazioni si traducono copiando la chiave primaria nella tabella collegata, una chiave di 4 attributi costringerà tutte le tabelle figlie ad avere 4 colonne aggiuntive solo per mantenere il legame. Questo allarga a dismisura le tabelle e spreca memoria.
2. **Costo Computazionale:** Gli indici sulle chiavi composite sono lenti da scorrere e pesanti da aggiornare, rendendo i JOIN estremamente inefficienti.

Per risolvere questo problema, si modifica il Class Diagram introducendo un nuovo attributo artificiale, privo di alcun significato nel dominio di business, chiamato **Chiave Surrogata** (o *Sintetica*). Generalmente si tratta di un numero intero auto-incrementante (es. `id`, `codice`) o di un UUID alfanumerico. Questo attributo viene eletto a nuova Chiave Primaria, sostituendo la chiave naturale composta.

Esempio 2.11 (Introduzione di una Chiave Surrogata). Si consideri la classe **Stanza** di un polo universitario. Nel mondo reale, una stanza è identificata univocamente dalla combinazione di tre attributi: `edificio`, `piano` e `numeroStanza`.

`STANZA(edificio, piano, numeroStanza, capienza)`

Se un corso dovesse essere assegnato a una stanza, lo schema **CORSO** dovrebbe ereditare tutte e tre le colonne come Foreign Key.

Ristrutturiamo lo schema introducendo una chiave surrogata `idStanza`:

`STANZA(idStanza, edificio, piano, numeroStanza, capienza)`

Ora, le tabelle collegate (come **CORSO**) dovranno importare solamente la colonna `idStanza` (un semplice intero) per stabilire l'associazione, ottimizzando drasticamente le prestazioni del database.

Attenzione alla perdita di integrità

Quando si sostituisce una chiave naturale con una chiave surrogata, il database userà il nuovo `id` per distinguere le righe. Questo crea un potenziale rischio: il database permetterà di inserire due righe con `id` diversi ma con gli **stessi identici dati naturali** (es. due stanze diverse assegnate allo stesso edificio, stesso piano e



stesso numero). Per evitare dati duplicati, è **obbligatorio** imporre un vincolo di univocità (UNIQUE) sull'insieme degli attributi che formavano l'originaria chiave naturale composita.

2.5.2 Analisi degli Attributi Derivati e delle Ridondanze

Durante la progettazione concettuale, è comune inserire attributi il cui valore non è indipendente, ma può essere ricavato matematicamente o logicamente da altri dati presenti nel sistema. Nel Class Diagram UML, questa caratteristica viene esplicitata anteponendo una barra (slash) al nome dell'attributo (es. /età).

Nel passaggio al Class Diagram revisionato (e successivamente allo schema logico), il progettista deve compiere una scelta architeturale precisa: l'attributo derivato verrà **effettivamente memorizzato** (storicizzato) nel database o verrà eliminato dallo schema e **calcolato a runtime** ogni volta che viene richiesto? Se si decide di non memorizzarlo, l'attributo scompare del tutto dallo schema revisionato.

Esempio 2.12 (Età vs Data di Nascita). Si consideri la classe *Persona* dotata degli attributi *dataDiNascita* ed /età. L'età è banalmente derivabile per differenza tra la data odierna e la data di nascita.

La scelta tra memorizzazione e calcolo on-the-fly comporta un compromesso tra prestazioni di lettura, spazio occupato e coerenza dei dati.

Scenario A: Memorizzazione (Storicizzazione del dato)

- **Vantaggi:** Tempi di risposta immediati durante le interrogazioni (query). Il DBMS si limita a leggere il dato senza sprecare cicli di CPU per eseguire calcoli, aspetto cruciale se il dato è richiesto frequentissimamente.
- **Svantaggi:**
 - Spreco di spazio su disco (memoria).
 - Rischio di *inconsistenza*: se la *dataDiNascita* viene corretta ma ci si dimentica di aggiornare l'età, il database conterrà informazioni contraddittorie.
 - Oneri di aggiornamento: dati altamente volatili (come l'età, che cambia ogni anno) richiedono procedure o *trigger* che aggiornino continuamente il database, generando un carico di scrittura pesante.

Scenario B: Calcolo a Runtime (Nessuna memorizzazione)

- **Vantaggi:** Nessun rischio di inconsistenza (il calcolo parte sempre dal "dato sorgente" aggiornato), risparmio di memoria e zero manutenzione per l'aggiornamento.
- **Svantaggi:** Costo computazionale pagato al momento dell'interrogazione. Se il calcolo è complesso e viene richiesto da milioni di utenti, il server potrebbe rallentare drasticamente.



Principi Generali per gli Attributi Derivati

Nell'ingegneria dei dati, la scelta si basa sull'analisi del carico di lavoro previsto e sulla volatilità del dato:

1. **Dati volatili e calcoli leggeri (Preferire il Calcolo):** Se il dato derivato cambia spesso nel tempo e l'operazione matematica è banale, l'attributo si elimina e si calcola al volo.
2. **Dati stabili e calcoli pesanti (Preferire la Memorizzazione):** Se il



dato originario, una volta inserito, non cambia più (es. il totale di una **Fattura** emessa e consolidata, ricavato dalla somma delle sue **Righe**), conviene storicizzare il totale. Il costo di aggiornamento è zero (la fattura non cambierà) e si evitano pesanti JOIN e somme ad ogni lettura.

Analisi delle Associazioni Ridondanti

Il concetto di ridondanza si estende oltre i singoli attributi e coinvolge l'intera topologia del Class Diagram. In uno schema complesso, potremmo accorgerci dell'esistenza di **associazioni derivabili** per transitività.

Ad esempio, se un **Impiegato** è associato a un **Dipartimento**, e un **Dipartimento** è associato a una **Città**, un'eventuale associazione diretta tra **Impiegato** e **Città** è da considerarsi un ciclo ridondante, poiché la città dell'impiegato è già ricavabile navigando il percorso **Impiegato** → **Dipartimento** → **Città**.

Come per gli attributi, il progettista deve decidere se mantenere l'associazione diretta (tramite l'aggiunta di una chiave esterna ridondante) o se farla collassare e affidarsi alla navigazione del grafo.



Principi Generali per le Associazioni Ridondanti

La gestione dei cicli si basa sul compromesso tra prestazioni di *routing* e integrità strutturale:

1. **Evitare anomalie strutturali (Preferire l'eliminazione):** La regola generale impone di rimuovere le associazioni ridondanti. Mantenere un legame diretto crea il rischio di inconsistenze: potremmo, per errore, assegnare l'Impiegato al Dipartimento di Milano, ma associarlo direttamente alla Città di Roma.
2. **Ottimizzazione estrema (Mantenere il legame diretto):** Si deroga alla regola precedente solo in sistemi ad altissimo traffico (*read-heavy*) in cui il percorso di navigazione è molto lungo (es. richiede di attraversare 4 o 5 tabelle con relative operazioni di JOIN). In quel caso, si introduce l'associazione ridondante come scorciatoia prestazionale, assumendosi l'onere (spesso tramite *stored procedures*) di mantenerla sincronizzata con il percorso principale.

2.5.3 Analisi delle Associazioni Uno a Uno: Fusione vs Separazione

Durante la progettazione concettuale, due entità legate da un'associazione rigorosamente biunivoca ($1..1 \leftrightarrow 1..1$) mantengono identità separate per ragioni semantiche. Poiché i loro cicli di vita coincidono (nascono e muoiono insieme), nella fase di ristrutturazione logica si presenta l'opportunità di **fondere le due classi in un unico schema relazionale**, consolidando i loro attributi ed eliminando la necessità di una chiave esterna.

Questa decisione architetturale comporta un delicato trade-off tra la velocità di lettura (assenza di JOIN) e l'occupazione di memoria (pesantezza della tabella), oltre a sollevare questioni di sicurezza dei dati.

Esempio 2.13 (Fusione vs Separazione: Paziente e Cartella Clinica). Si consideri l'associazione obbligatoria ($1..1 \leftrightarrow 1..1$) tra un **Paziente** e la sua **CartellaClinica**. Ogni paziente ha una sola cartella clinica, e ogni cartella appartiene a un solo paziente.

Scenario A: Fusione (Unico Schema) Se fondessimo le due entità, otterremmo un'unica tabella massiccia:

PAZIENTE(codiceFiscale, nome, telefono, gruppoSanguigno, anamnesiCompleta)

- *Vantaggio*: Nessun JOIN necessario per leggere i dati completi.
- *Svantaggio*: L'attributo **anamnesiCompleta** potrebbe essere un testo lunghissimo (BLOB o TEXT). Ogni volta che un impiegato dell'accettazione interroga il database solo per cercare il numero di **telefono** del paziente, il motore del database è costretto a caricare in memoria dal disco fisso anche i megabyte di testo dell'anamnesi, sprecando enormi risorse di I/O. Inoltre, si espongono dati sensibili a personale non medico.

Scenario B: Separazione (Due Schemi) Mantenendo gli schemi separati, applichiamo un *partizionamento verticale* logico:

- PAZIENTE(codiceFiscale, nome, telefono)
- CARTELLA_CLINICA(codiceCartella, gruppoSanguigno, anamnesi, cfPaziente)

Vantaggi:

1. **Performance**: La tabella PAZIENTE è "leggera" e stretta. Le query anagrafiche quotidiane saranno fulminee e il database potrà tenere l'intera tabella nella cache RAM.
2. **Sicurezza**: Si possono impostare permessi di accesso (GRANT) a livello di tabella, permettendo alla segreteria di leggere PAZIENTE e impedendo l'accesso a CARTELLA_CLINICA, riservato ai soli medici.

Svantaggio: Quando un medico avrà bisogno del nome del paziente e della sua anamnesi contemporaneamente, il sistema dovrà pagare il costo computazionale di un JOIN.



La Regola Pratica

La fusione in un unico schema è raccomandata quando gli attributi della seconda classe sono pochi, di piccole dimensioni (es. interi, date, stringhe brevi) e vengono interrogati quasi sempre insieme a quelli della classe principale (alta coesione). La separazione è invece mandatoria in presenza di attributi "pesanti" (es. file, immagini, testi lunghi) o sottoposti a stringenti policy di privacy.

2.5.4 Analisi degli Attributi Strutturati

Nel diagramma delle classi originario, è frequente l'utilizzo di **attributi strutturati** (o *composti*), ovvero attributi che raggruppano al loro interno campi di natura eterogenea. L'esempio classico è l'attributo **residenza** assegnato a una classe **Persona**, il quale è concettualmente composto da **via**, **civico**, **cap** e **città**.

Il Modello Relazionale, tuttavia, impone il rispetto della **Prima Forma Normale (1NF)**, la quale stabilisce che il dominio di ogni attributo debba essere **atomico** (indivisibile). I database relazionali puri non supportano l'inserimento di "sotto-strutture" o "oggetti" all'interno di una singola cella.

Per risolvere questa incompatibilità durante la fase di ristrutturazione, il progettista deve eliminare l'attributo strutturato scegliendo una delle seguenti tre strategie:

1. Appiattimento (Flattening) degli attributi

L'attributo strutturato viene rimosso e sostituito dall'esplicitazione di tutti i suoi sotto-campi direttamente come colonne della classe principale.

- **Schema:** PERSONA(cf, nome, via, civico, cap, città)
- **Vantaggi:** Non richiede JOIN. Permette di effettuare ricerche efficienti sui singoli sotto-campi (es. "trova tutte le persone che abitano a Roma").
- **Svantaggi:** "Allarga" la tabella principale, rischiando di riempirla di valori NULL qualora l'attributo strutturato originario fosse opzionale (0..1).

2. Accorpamento in formato Stringa (Concatenazione)

I sotto-campi vengono fusi in un'unica stringa testuale, inserita in un solo attributo atomico.

- **Schema:** PERSONA(cf, nome, indirizzoCompleto)
- **Vantaggi:** Estrema semplicità e compattezza dello schema.
- **Svantaggi:** Rende difficilissime o ineseguibili le query analitiche (es. contare le persone per CAP richiederebbe complesse e lente operazioni di *parsing* testuale tramite LIKE o *Regex*).

3. Creazione di un'Entità Autonoma

Si estrapola l'attributo strutturato promuovendolo a nuova **Classe autonoma**, collegata a quella originale tramite un'associazione.

- **Schema:** PERSONA(..., idResidenza) → RESIDENZA(id, via, civico, città)
- **Vantaggi:** Massima pulizia formale. Permette di riutilizzare l'entità (es. se cinque membri di una famiglia condividono la stessa residenza, il dato non viene duplicato 5 volte, ma inserito una volta sola e linkato).
- **Svantaggi:** Introduce un costo computazionale (JOIN) per ogni lettura completa e rende più complessa la fase di inserimento dei dati.

Principi Generali di Scelta

La decisione tra queste tre soluzioni è dettata dall'utilizzo pratico che l'applicazione farà del dato:

1. **Se il dato è un blocco opaco (Preferire l'Accorpamento testuale):** Se l'indirizzo serve esclusivamente per essere stampato su un'etichetta di spedizione e al sistema non interesserà mai fare statistiche per città o per CAP, una stringa formattata è la scelta più leggera e sensata.
2. **Se i campi hanno dignità di interrogazione (Preferire l'Appiattimento):** Se il software prevede filtri di ricerca (es. un menù a tendina per filtrare gli utenti per Provincia), i campi devono essere separati per poter essere indicizzati singolarmente.
3. **Se l'attributo ha vita propria o cardinalità multipla (Preferire l'Entità Autonoma):** Se l'indirizzo deve essere storicizzato (tenere traccia dei vecchi indirizzi), se una persona ha molteplici indirizzi (casa, lavoro, fatturazione), o se lo stesso indirizzo raggruppa logicamente più utenti (come in un'azienda), l'attributo strutturato **deve** diventare una tabella separata.

2.5.5 Analisi degli Attributi a Valore Multiplo

Nel Class Diagram UML è perfettamente lecito assegnare a un'entità un attributo con cardinalità multipla, ovvero una lista o un array di valori (es. `telefoni [1..*]` per la classe `Persona`). Tuttavia, il modello relazionale ammette unicamente **valori singoli e scalari** per ogni cella. L'inserimento di vettori o array non è nativamente supportato dalle regole dell'algebra relazionale.

Il tentativo di risolvere il problema "appiattendo" l'attributo in una serie di colonne fisse (`telefono1`, `telefono2`, ...) rappresenta una pessima pratica ingegneristica: spreca enormi quantità di memoria (generando celle NULL per gli utenti con un solo recapito), limita la scalabilità (non permette l'inserimento di un numero di recapiti superiore alle colonne predisposte) e rende le query di ricerca estremamente inefficienti.

Per normalizzare lo schema, il progettista deve eliminare l'attributo multivalore ricorrendo a una delle seguenti soluzioni:

1. Accorpamento testuale (Formattazione Custom)

Tutti i valori dell'array vengono concatenati e salvati in un'unica stringa di testo, separati da un carattere speciale (es. virgola o punto e virgola).

- **Schema:** `PERSONA(cf, nome, listaTelefoni)`
- *Valore di esempio:* "3331234567; 069876543"
- **Vantaggi:** Non modifica la struttura del database e non richiede tabelle aggiuntive.
- **Svantaggi:** Impedisce al database di effettuare controlli sui singoli valori (es. non si può usare un vincolo `UNIQUE` per impedire che due utenti inseriscano lo stesso numero). Rendere aggiornabile o ricercabile un singolo numero richiede complesse operazioni di manipolazione delle stringhe.

2. Creazione di una Classe Esterna (Associazione 1:N)

Si estrapola l'attributo trasformandolo in un'entità autonoma debole, legata all'entità principale da un'associazione uno a molti. È la soluzione relazionale per eccellenza.

- **Schema:**
 - `PERSONA(cf, nome)`
 - `TELEFONO(numero, cfPersona)`
- **Vantaggi:** Scalabilità infinita (una persona può avere da zero a mille telefoni senza sprecare byte). Le query sono pulite e veloci, e si possono imporre vincoli di integrità sui singoli numeri.
- **Svantaggi:** Richiede un'operazione di `JOIN` per estrarre l'elenco completo dei telefoni associati a una persona.

Principi Generali di Scelta

La scelta della tecnica di risoluzione è quasi sempre a favore dell'entità esterna, tranne in casi specifici di puro logging:

1. **Dati operativi e ricercabili (Regola Aurea → Entità Esterna):** Se il dato ha valore di business, se il sistema dovrà mai cercare "a chi appartiene questo numero", se l'utente deve poter modificare o cancellare un singolo valore dalla sua lista, la creazione di una tabella esterna 1:N è l'unica via ingegneristicamente solida.



2. **Dati descrittivi morti (Eccezione → Accorpamento Testuale):** Si opta per la stringa formattata solo per dati passivi, che vengono scritti una volta e letti in blocco unicamente per stamparli a schermo. Ad esempio: un attributo multivalore `note_aggiuntive` o `tag_ricerca` in sistemi molto semplici.

2.5.6 Codifica delle Gerarchie (Ereditarietà)

Il modello relazionale è intrinsecamente "piatto" e non supporta nativamente il concetto orientato agli oggetti di ereditarietà (superclasse e sottoclassi). Pertanto, quando nel Class Diagram si incontra una gerarchia di generalizzazione/specializzazione, è necessario "smontarla" trasformandola in tabelle ordinarie.

L'ingegneria del software offre tre strategie principali per affrontare questa traduzione, ciascuna con precisi impatti su prestazioni, memoria e integrità dei dati.

1. Partizionamento Orizzontale (Accorpamento verso il basso)

Questa soluzione prevede l'eliminazione della superclasse e la creazione di una tabella per ciascuna sottoclasse. Ogni tabella figlia "eredita" fisicamente copiando al proprio interno tutti gli attributi della generalizzazione.

- **Schema logico:** `PERSONA(nome, cognome)` scompare. Si creano `STUDENTE(id, nome, cognome, matricola)` e `DOCENTE(id, nome, cognome, stipendio)`.
- **Vantaggi:** Interrogare una specifica sottoclasse è estremamente veloce (nessun JOIN).
- **Svantaggi e Limiti:** Questa strada è matematicamente percorribile **solo se la generalizzazione è totale** (ovvero se non esistono istanze della superclasse che non appartengano a nessuna sottoclasse). Inoltre, se si vuole fare una query su tutte le persone (indipendentemente dal tipo), il database deve eseguire una pesante operazione di UNION tra le tabelle.

2. Partizionamento Singolo (Accorpamento verso l'alto)

Questa soluzione, all'opposto, prevede l'eliminazione delle sottoclassi. Si crea un'unica, grande tabella (la superclasse) che ingloba tutti gli attributi di tutte le specializzazioni, introducendo un attributo **discriminante** (un enumeratore) per capire di che "tipo" sia la riga.

- **Schema logico:** `PERSONA(id, tipoPersona, nome, cognome, matricola, stipendio)`
- **Vantaggi:** È sempre applicabile ed è la soluzione più performante in assoluto (niente JOIN, niente UNION). Molto amata nello sviluppo web moderno per la sua velocità in lettura.
- **Svantaggi (Problema dei vincoli persi):** Genera tabelle "sparse", piene di valori NULL (uno studente avrà lo stipendio NULL). Soprattutto, fa perdere il rigore delle associazioni originarie.

Esempio 2.14 (Recupero dei vincoli di consistenza). Nel Class Diagram, supponiamo che l'associazione *"Insegna"* leghi la classe **Corso** ESCLUSIVAMENTE alla sottoclasse **Docente**. Utilizzando l'accorpamento verso l'alto, la tabella **CORSO** dovrà avere una chiave esterna che punta all'unica tabella rimasta: **PERSONA**. Questo permette a livello strutturale l'assurdità di far insegnare un corso a uno Studente. Per recuperare l'informazione persa, si

deve imporre un vincolo applicativo o un *CHECK constraint* SQL che assicuri che la chiave esterna punti solo a righe in cui l'attributo discriminante `tipoPersona = 'Docente'`.

3. Partizionamento Verticale (Traduzione in Associazioni 1:1)

Questa strategia preserva la struttura del diagramma originale. Si crea una tabella per la superclasse (con i soli attributi comuni) e una tabella per ogni sottoclasse (con i soli attributi specifici). Le tabelle figlie vengono legate alla tabella padre tramite un'associazione 1:1 rigorosa, utilizzando la tecnica della **Chiave Primaria Condivisa**.

- **Schema logico:**
 - PERSONA(idPersona, nome, cognome)
 - STUDENTE(idPersona, matricola)
 - DOCENTE(idPersona, stipendio)
- **Vantaggi:** Rispetta la terza forma normale, non spreca spazio con i NULL e mantiene perfettamente validi i vincoli delle associazioni. Nel caso di **ereditarietà disgiunta**, è necessario aggiungere un *trigger* che impedisca allo stesso `idPersona` di comparire sia in STUDENTE che in DOCENTE.
- **Svantaggi:** È la soluzione più costosa a livello computazionale. L'estrazione dei dati completi di un utente richiederà sempre un JOIN tra la tabella specifica e la superclasse.

Principi Generali di Scelta dell'Ereditarietà

Non esiste una soluzione perfetta a priori, ma la pratica ingegneristica suggerisce queste euristiche:

1. **Usa l'accorpamento in alto (Singola Tabella):** Quando le sottoclassi differiscono per pochissimi attributi e non hanno relazioni specifiche pesanti. Il piccolo spreco di spazio dovuto ai NULL è ampiamente ripagato dalla velocità estrema delle query.
2. **Usa l'accorpamento in basso (Tabelle Separate):** Quando la generalizzazione è totale, le query non cercano quasi mai la "superclasse" generica, e le sottoclassi sono entità molto ricche e complesse di per sé.
3. **Usa le associazioni 1:1 (Tabella per Classe):** Quando l'integrità dei dati è la priorità assoluta, le sottoclassi hanno tantissime associazioni specifiche con altre tabelle del database e lo spazio su disco deve essere ottimizzato al massimo.

2.6 Riepilogo: Il Ciclo di Vita della Progettazione dei Dati

Prima di introdurre il linguaggio di interrogazione e definizione dei dati, è utile schematizzare l'intera metodologia di progettazione di basi di dati affrontata finora. Il processo ingegneristico segue un flusso di raffinamento progressivo (dal livello astratto al livello implementativo), suddiviso in fasi distinte e sequenziali. Il quarto punto rappresenta un'anticipazione di quanto vedremo nel prossimo capitolo.

1. Progettazione Concettuale (Modellazione del Dominio)

- *Obiettivo:* Rappresentare la semantica della realtà di interesse in modo puro e indipendente dalla tecnologia.
- *Artefatti prodotti:* **Class Diagram UML** iniziale.

- *Documentazione a corredo*: **Dizionari dei Dati** (dizionario delle classi, dizionario delle associazioni e dizionario dei vincoli di business non esprimibili graficamente).
2. **Ristrutturazione dello Schema (Revisione Concettuale)**
 - *Obiettivo*: Adattare il modello concettuale puro ai limiti e ai requisiti del modello logico di destinazione (es. eliminazione attributi strutturati/multivalore, gestione delle gerarchie, ottimizzazione delle chiavi).
 - *Artefatti prodotti*: **Class Diagram Revisionato**.
 - *Documentazione a corredo*: Aggiornamento dei dizionari. In particolare, il **Dizionario dei Vincoli** subisce un forte arricchimento per recuperare semanticamente tutte le informazioni perse durante gli appiattimenti strutturali (es. vincoli per le ereditarietà accorpate).
 3. **Progettazione Logica (Traduzione Relazionale)**
 - *Obiettivo*: Tradurre gli elementi del diagramma revisionato in strutture dati compatibili con l'algebra relazionale.
 - *Artefatti prodotti*: **Schemi Relazionali** definitivi, con esplicita indicazione degli identificatori (*Chiavi Primarie* - *PK*) e dei legami di navigabilità (*Chiavi Esterne* - *FK*).
 4. **Implementazione e Definizione dei Metadati (Verso il livello Fisico)**
 - *Obiettivo*: Istruire materialmente il motore del database (DBMS) a creare le strutture logiche appena progettate.
 - *Strumento*: Utilizzo del linguaggio **SQL** (in particolare il sotto-insieme DDL, *Data Definition Language*), attraverso il quale gli schemi relazionali e i vincoli vengono codificati nei metadati del sistema.

Introduzione a SQL e Definizione dei Metadati

3.1 Nozioni introduttive

Conclusa la fase di progettazione logica, il progettista ha tra le mani uno schema relazionale formale e matematicamente inattaccabile. Tuttavia, questo schema risiede ancora su carta. Il passo successivo consiste nell'istanziare fisicamente questo modello all'interno di un **DBMS** (Database Management System, come PostgreSQL, MySQL o Oracle).

Per comunicare con il motore del database si utilizza **SQL (Structured Query Language)**, il linguaggio standard universale per la gestione dei database relazionali.

Lo Standard SQL e la Portabilità

Sebbene SQL sia il linguaggio standard per i database relazionali (codificato dagli enti internazionali ANSI e ISO), nella realtà commerciale ogni motore DBMS (PostgreSQL, MySQL, Oracle, ecc.) implementa un proprio "dialetto", introducendo estensioni e funzioni proprietarie non previste dallo standard.

Per questo motivo, il manuale ufficiale di un buon DBMS inizia solitamente dichiarando esplicitamente il proprio livello di aderenza allo standard internazionale. Dal punto di vista dell'ingegneria del software, scrivere codice il più aderente possibile allo standard puro è una *best practice* fondamentale: assicura un'elevata **portabilità** del sistema, garantendo la possibilità di migrare l'infrastruttura da un DBMS a un altro con sforzi e costi di riscrittura minimi. Nel corso della progettazione si farà riferimento esclusivamente alle direttive dell'SQL standard.

Il concetto di Metadato e il Catalogo di Sistema

Prima di poter inserire i dati reali dell'applicazione (es. Mario Rossi, matricola 12345), il DBMS deve conoscere la "forma" che questi dati avranno, le regole a cui devono sottostare e i legami che li uniscono. Queste informazioni strutturali prendono il nome di **Metadati** (letteralmente: *dati che descrivono altri dati*).

Quando si definiscono gli schemi e i vincoli in SQL, il motore del database non sta ancora salvando informazioni applicative, ma sta popolando il proprio **Catalogo di Sistema** (o *Data Dictionary*). Il catalogo è un set di tabelle interne, invisibili all'utente finale, che il DBMS usa per ricordare quante tabelle esistono, come si chiamano le colonne, quali sono i tipi di dato ammessi e quali vincoli di chiave primaria o esterna devono essere imposti.

L'Anatomia di SQL: Un Linguaggio Dichiarativo

A differenza di linguaggi imperativi come Java, C o Python, in cui il programmatore deve descrivere *passo dopo passo* come eseguire un'operazione, SQL è un linguaggio **dichiarativo**: il progettista si limita a dichiarare *cosa* vuole ottenere o *come* deve essere fatta la struttura; sarà il motore di ottimizzazione interno del DBMS a decidere la strategia algoritmica migliore per allocare la memoria o estrarre i dati.

Per gestire l'intero ciclo di vita del dato, il linguaggio SQL è suddiviso in tre grandi sotto-insiemi funzionali (sub-languages):

1. **DDL (Data Definition Language):** È il linguaggio di definizione dei metadati. Viene utilizzato esclusivamente per manipolare la struttura del database (il contenitore). I suoi comandi principali sono:
 - **CREATE:** per generare nuove tabelle, indici o viste.
 - **ALTER:** per modificare la struttura di una tabella esistente (es. aggiungere una colonna).
 - **DROP:** per distruggere fisicamente una struttura e i suoi dati.
2. **DML (Data Manipulation Language):** È il linguaggio utilizzato per manipolare il contenuto (i dati veri e propri) all'interno delle strutture precedentemente create col DDL. I comandi principali sono:
 - **INSERT:** per inserire nuove tuple.
 - **UPDATE:** per aggiornare i valori di tuple esistenti.
 - **DELETE:** per rimuovere tuple.
3. **DQL (Data Query Language):** Spesso considerato parte del DML, è l'istruzione **SELECT**, il cuore pulsante dei database relazionali, utilizzata per interrogare il motore e filtrare, aggregare o unire (**JOIN**) i dati.

3.2 I Domini di Base (Tipi di Dato) nello Standard SQL

Nella creazione di una tabella tramite DDL, ogni attributo deve essere obbligatoriamente associato a un **Dominio** (tipo di dato). Il dominio definisce lo spazio dei valori ammissibili per quella colonna, lo spazio fisico occupato su disco e le operazioni matematiche o logiche consentite.

Lo Standard SQL classifica i tipi di dato in grandi famiglie. Di seguito vengono presentati i domini fondamentali e maggiormente utilizzati.

Dati di tipo Stringa (Caratteri)

Servono per memorizzare testo, codici alfanumerici o numeri su cui non si devono effettuare operazioni matematiche (es. numeri di telefono, CAP, codici fiscali).

- **CHAR(n):** Stringa a lunghezza **fissa** pari a n caratteri. Se si inserisce una stringa più corta, il DBMS la "riempie" (padding) con spazi vuoti fino a raggiungere n .
 - *Uso ingegneristico:* Estremamente veloce per la ricerca. Si usa **solo** quando il dato ha lunghezza rigidamente costante (es. **CHAR(16)** per il Codice Fiscale, **CHAR(2)** per la sigla della Provincia).
- **VARCHAR(n):** Stringa a lunghezza **variabile** con limite massimo di n caratteri. Se si inserisce una stringa più corta, occupa su disco solo lo spazio effettivo più un marcatore di fine stringa.
 - *Uso ingegneristico:* Lo standard *de facto* per quasi tutto il testo (es. **VARCHAR(50)** per il Nome, **VARCHAR(255)** per l'Email). Fa risparmiare enorme spazio su disco, a costo di una microscopica latenza in più durante l'allocazione della memoria.

Dati di tipo Numerico

SQL distingue rigorosamente tra numeri esatti (dove $1.1 + 1.1$ fa esattamente 2.2) e numeri approssimati (soggetti a virgola mobile e possibili errori di precisione).

- **Numerici Esatti Interi:**

- **INT** o **INTEGER**: Numero intero standard con segno (solitamente a 32 bit, range da circa -2 a +2 miliardi). È il re indiscusso delle chiavi primarie surrogate e dei contatori.
- **SMALLINT**: Numero intero "piccolo" (solitamente a 16 bit). Utile per risparmiare RAM se il range di valori è sicuramente ridotto (es. l'anno di immatricolazione, o valori da 1 a 100).
- **Numerici Decimali Esatti**:
 - **NUMERIC(p, s)** o **DECIMAL(p, s)**: Numeri a virgola fissa. Il parametro *p* (*precision*) indica il numero totale di cifre, mentre *s* (*scale*) indica quante di queste cifre stanno dopo la virgola.
 - *Esempio*: **NUMERIC(5, 2)** può memorizzare valori da -999.99 a 999.99.
 - *Uso ingegneristico*: Obbligatorio per i dati **finanziari e monetari** (stipendi, prezzi, bilanci). Garantisce che non ci siano mai arrotondamenti imprevisti.
- **Numerici Approssimati**:
 - **REAL, DOUBLE PRECISION, FLOAT**: Numeri a virgola mobile (standard IEEE 754).
 - *Uso ingegneristico*: Si usano **solo** per calcoli scientifici, coordinate GPS o misurazioni fisiche dove la scala è immensa e una micro-approssimazione (es. 1.9999999 al posto di 2) è tollerabile. Da evitare assolutamente per la valuta.

Dati di tipo Temporale

- **DATE**: Memorizza solo la data (Anno, Mese, Giorno). Formato: 'YYYY-MM-GG'. Ottimo per date di nascita o date di assunzione.
- **TIME**: Memorizza solo l'orario (Ore, Minuti, Secondi). Formato: 'HH-MI-SS'.
- **TIMESTAMP** o **DATETIME**: Memorizza data e ora esatte in un unico campo. Formato: 'YYYY-MM-GG HH-MI-SS'. Fondamentale per i log di sistema, la data di creazione di un record o per tracciare transazioni.

La Rappresentazione Fisica dei Tipi Temporal

Nonostante in fase di interrogazione e inserimento le date vengano espresse in formato testuale (es. 'YYYY-MM-DD'), a livello di immagazzinamento fisico i tipi temporali **non sono stringhe**.

Il DBMS analizza la stringa immessa e la converte in un numero (spesso un intero a 32 o 64 bit che rappresenta i giorni o i millisecondi trascorsi da una specifica data di riferimento, detta *Epoch*). Questa astrazione matematica è cruciale per tre motivi:

1. **Performance aritmetica**: Permette al DBMS di sommare giorni o calcolare intervalli tramite velocissime operazioni matematiche a livello di CPU, evitando il costoso *parsing* testuale.
2. **Ottimizzazione spaziale**: Un **DATE** occupa tipicamente 4 byte, contro i 10 byte di un **CHAR(10)**.
3. **Validazione semantica**: Il DBMS controlla nativamente l'esistenza reale della data (es. impedendo l'inserimento del 30 Febbraio).

Dati di tipo Booleano

- **BOOLEAN**: Ammette solo i valori di verità **TRUE** o **FALSE** (più il valore **NULL** se il campo non è obbligatorio). Ottimo per i flag (es. **is_attivo**, **is_pagato**).

⚠ Il NULL non è un tipo di dato

È fondamentale ricordare che NULL non appartiene a nessun dominio specifico, ma è uno **stato** universale. NULL in SQL non significa "zero" né "stringa vuota", ma rappresenta "informazione mancante, sconosciuta o inapplicabile"; intuitivamente, si può pensare come una variabile incognita (come in una equazione matematica). Qualsiasi tipo di dato descritto sopra, in assenza di vincoli, può assumere lo stato NULL.

🔍 Il supporto al tipo BOOLEAN e l'approccio legacy

Sebbene lo standard SQL preveda nativamente il dominio **BOOLEAN**, nella pratica ingegneristica è necessario tenere conto delle limitazioni specifiche di alcuni DBMS. Il caso storico più celebre è Oracle Database, che non ha supportato il tipo booleano a livello di tabella fino alla versione 23c (2023).

Quando si opera su sistemi legacy o DBMS privi di questo dominio, la *best practice* architetturale consiste nel "simulare" il booleano utilizzando un tipo di dato minimale, come **CHAR(1)** o **SMALLINT/NUMBER(1)**, e "blindarlo" tramite un vincolo di colonna personalizzato (il vincolo **CHECK**, che vedremo a breve) che limiti esplicitamente l'inserimento a due soli valori convenzionali.

3.3 Operatori Logici e di Confronto

Per imporre regole di business (tramite vincoli **CHECK**) o per filtrare i dati durante le interrogazioni, SQL utilizza espressioni condizionali. Un'espressione condizionale valuta il contenuto di una cella e restituisce un risultato booleano a tre vie: **TRUE**, **FALSE** oppure **UNKNOWN** (se coinvolge un valore NULL).

Di seguito i principali operatori standard previsti dal linguaggio:

Operatori di Confronto Standard

Si applicano a stringhe, numeri e date:

- **=** (Uguale)
- **<>** oppure **!=** (Diverso)
- **<**, **>**, **<=**, **>=** (Minore, Maggiore e relativi uguali)

Operatori Logici (Algebra di Boole)

Permettono di combinare più condizioni elementari:

- **AND**: Restituisce **TRUE** solo se *tutte* le condizioni combinate sono vere.
- **OR**: Restituisce **TRUE** se *almeno una* delle condizioni è vera.
- **NOT**: Inverte il risultato logico della condizione che lo segue.

Operatori Speciali di SQL

Sono costrutti sintattici avanzati che semplificano enormemente la scrittura del codice, ottimizzando le performance del motore di ricerca:

- **BETWEEN x AND y**: Verifica se un valore è compreso in un intervallo continuo (inclusi gli estremi x e y). Ottimo per le date o i range numerici.
 - *Esempio*: voto **BETWEEN 18 AND 30** equivale a voto **>= 18 AND voto <= 30**.

- **IN** (*val1*, *val2*, ...): Verifica se un valore appartiene a una lista di elementi discreti.
 - *Esempio*: `provincia IN ('RM', 'MI', 'NA')` equivale a `provincia = 'RM' OR provincia = 'MI' OR provincia = 'NA'`.
- **LIKE** '*pattern*': Permette il confronto testuale parziale (pattern matching) sulle stringhe. Utilizza due caratteri jolly (wildcard):
 - `%` (Percentuale): Sostituisce una sequenza di zero o più caratteri.
 - `_` (Underscore): Sostituisce esattamente un singolo carattere.
 - *Esempio*: `cognome LIKE 'R%'` (Trova tutti i cognomi che iniziano per R); `nome LIKE '_ario'` (Trova Mario, Dario, ma non Ilario).

⚠ La trappola del NULL: IS NULL vs

Per verificare se una cella è vuota, l'intuito del programmatore suggerirebbe di scrivere `colonna = NULL`. In SQL, questa sintassi è **errata** e restituirà sempre UNKNOWN, facendo fallire la condizione. Poiché NULL rappresenta un valore "incognito", è impossibile affermare matematicamente che un'incognita sia "uguale" a un'altra incognita.

L'unico modo legittimo in SQL per verificare l'assenza o la presenza di un dato è utilizzare gli operatori specifici di stato:

- **IS NULL** (Verifica se il campo è vuoto)
- **IS NOT NULL** (Verifica se il campo contiene un valore)

3.4 Istruzioni DDL

Il Data Definition Language (DDL) permette di definire la struttura fisica del database. I due comandi fondamentali per istanziare l'architettura dei dati sono **CREATE SCHEMA** e **CREATE TABLE**.

✓ Convenzione Stilistica (Case Sensitivity)

Lo standard SQL è *case-insensitive* per quanto riguarda le parole chiave del linguaggio. Tuttavia, è convenzione universale nell'ingegneria del software scrivere le keyword in **MAIUSCOLO** (es. `CREATE TABLE`, `VARCHAR`) e gli identificatori definiti dall'utente in **minuscolo** (es. `studenti`, `matricola`). Questo garantisce la massima leggibilità del codice.

3.4.1 Creazione di uno schema logico

CREATE SCHEMA crea quello che in informatica viene chiamato un *namespace* o, più semplicemente, un contenitore logico (un "recinto"). Serve a raggruppare un insieme di tabelle, viste e vincoli che appartengono alla stessa applicazione, separandoli da altri progetti presenti nello stesso server database. È importante notare che una tabella non può fluttuare nel vuoto: deve sempre risiedere all'interno di uno schema. Se non si specifica uno schema personalizzato con **CREATE SCHEMA**, molti DBMS inseriscono automaticamente le tabelle in uno schema di default chiamato comunemente **public** o **dbo**.

La sintassi di base è:

```
1 CREATE SCHEMA <nome_schema>;
```

3.4.2 Creazione di una tabella

Il comando `CREATE TABLE` istanzia fisicamente uno schema relazionale $S(A_1, \dots, A_k)$ associando a ciascun attributo A_i il proprio dominio $dom(A_i)$. Il nome assegnato alla tabella deve essere **unico** all'interno dello schema (o del database, se non si usano schemi espliciti).

La struttura del comando prevede una coppia di parentesi tonde all'interno delle quali si susseguono istruzioni separate da virgole.

Esempio 3.1 (Creazione tabella senza vincoli espliciti). Di seguito si riporta la traduzione in SQL dello schema relazionale relativo agli studenti. In questa prima fase esplorativa, ci limitiamo a definire i nomi degli attributi e i rispettivi domini (tipi di dato), omettendo temporaneamente i vincoli (come la chiave primaria).

```

1 CREATE TABLE studenti (
2     matricola CHAR(9),
3     cf        CHAR(16),
4     nome      VARCHAR(20),
5     cognome   VARCHAR(20),
6     dataN     DATE
7 );

```

Logicamente, il corpo del comando si divide in due blocchi sequenziali:

1. **Definizione degli Attributi:** In questa prima fase si elencano le colonne. Per ogni colonna si deve specificare il nome, il tipo di dato e, opzionalmente, dei vincoli specifici applicabili alla singola cella (vincoli intra-attributo).
2. **Definizione dei Vincoli di Tabella:** In questa seconda fase (che vedremo subito a seguire), si esplicitano i vincoli complessi che coinvolgono più colonne contemporaneamente (come chiavi primarie composte o chiavi esterne).

La sintassi per la definizione di un singolo attributo (la riga base) è la seguente:

`<nome_attributo> <tipo_attributo> [vincoli_attributo]`

Si noti come la scelta dei domini rispecchi quanto discusso nel paragrafo precedente:

- `matricola` e `cf` utilizzano `CHAR` poiché hanno una lunghezza fissa e rigidamente definita (rispettivamente 9 e 16 caratteri).
- `nome` e `cognome` utilizzano `VARCHAR` per ottimizzare lo spazio su disco, dato che la lunghezza reale del testo inserito è variabile.
- `dataN` (data di nascita) utilizza il tipo nativo `DATE` per garantire validazione e correttezza semantica.

3.4.3 I Vincoli Intra-attributo (o in-line)

I vincoli in-line sono regole di integrità applicate direttamente alla definizione di un singolo attributo, sulla stessa riga in cui viene dichiarato il tipo di dato. Servono a limitare i valori ammissibili nella singola cella, agendo come prima barriera contro la corruzione dei dati.

Lo Standard SQL prevede quattro vincoli principali in-line:

NOT NULL (Obbligatorietà)

Indica che il campo non può mai assumere lo stato `NULL`. All'atto dell'inserimento di una nuova riga, l'utente è obbligato a fornire un valore valido per questa colonna.

- *Uso:* Si applica a tutti i dati vitali che definiscono l'entità (es. il cognome di una persona, il titolo di un libro).

UNIQUE (Univocità di colonna)

Impone che tutti i valori presenti in quella colonna siano distinti tra loro. Il DBMS rifiuterà qualsiasi tentativo di inserire una riga con un valore già esistente in un'altra riga per quel campo.

- *Uso:* Si applica alle chiavi naturali candidate (es. il Codice Fiscale, l'Email). Nello standard SQL, a meno di specifiche implementazioni dei singoli DBMS, una colonna **UNIQUE** consente l'inserimento di molteplici valori **NULL**, poiché **NULL** rappresenta una incognita e due incognite non sono considerate uguali tra loro.

PRIMARY KEY (Chiave Primaria a livello di colonna)

Elegge l'attributo a identificatore principale della tabella. Specificare **PRIMARY KEY** su una colonna implica contemporaneamente i vincoli **NOT NULL** e **UNIQUE**. Una tabella può avere **una sola** chiave primaria.

- *Uso:* Si usa quando la chiave primaria è composta da un solo attributo (es. una chiave surrogata come `id` o una chiave naturale singola come `matricola`).

DEFAULT (Valore predefinito)

Non è un vincolo di integrità in senso stretto, ma una direttiva di assegnazione. Se durante un inserimento (**INSERT**) l'utente omette il valore per questa colonna, il DBMS vi inserisce automaticamente il valore specificato dopo la keyword **DEFAULT**.

- *Uso:* Ottimo per impostare stati iniziali (es. `is_attivo BOOLEAN DEFAULT TRUE`) o timestamp di creazione (es. `data_inserimento DATE DEFAULT CURRENT_DATE`).

Esempio 3.2 (Evoluzione dello schema studenti con vincoli di colonna). Riprendiamo lo schema della tabella `studenti` e applichiamo i vincoli intra-attributo per blindarne la coerenza semantica ed evitare anomalie:

```

1 CREATE TABLE studenti (
2     matricola CHAR(9) PRIMARY KEY,
3     cf        CHAR(16) UNIQUE NOT NULL,
4     nome      VARCHAR(20) NOT NULL,
5     cognome   VARCHAR(20) NOT NULL,
6     dataN     DATE
7 );

```

Analisi delle scelte effettuate:

- **matricola:** È l'identificatore principale. Diventa **PRIMARY KEY**, quindi sarà univoca e obbligatoria per definizione.
- **cf:** Il codice fiscale deve essere univoco (**UNIQUE**) per evitare duplicati nella realtà, ma deve anche essere obbligatorio (**NOT NULL**) per ogni studente registrato.
- **nome e cognome:** Viene applicato **NOT NULL** perché un'istanza di studente priva di identità anagrafica non avrebbe alcun significato logico.
- **dataN:** Rimane priva di vincoli espliciti; questo significa che il sistema accetterà il valore **NULL** qualora la data di nascita non fosse momentaneamente disponibile al momento della registrazione.

3.4.4 I Vincoli di Tabella (Out-of-line)

Fino a questo momento abbiamo visto i vincoli applicati direttamente sulla stessa riga della definizione dell'attributo (sintassi *in-line*). Tuttavia, lo standard SQL permette (e in molti casi impone) di dichiarare i vincoli in una sezione dedicata alla fine del comando `CREATE TABLE`, separata dalla definizione delle colonne. Questa formulazione prende il nome di **Vincolo di Tabella** (o *out-of-line*).

La sintassi generale per dichiarare un vincolo a livello di tabella è:

```
CONSTRAINT <nome_vincolo> <TIPO_VINCOLO> (colonna1, colonna2, ...)
```

Sebbene la clausola `CONSTRAINT <nome>` sia opzionale (se omessa, il DBMS genera internamente un nome alfanumerico casuale, es. `SYS_C00142`), nella pratica si raccomanda di assegnare sempre un nome esplicito e semantico (es. `pk_studenti` o `unq_cf`).

Nominare i vincoli è fondamentale per due operazioni architetturali:

1. **Leggibilità degli errori:** Se un'applicazione viola una regola, il database genererà un'eccezione specificando il nome del vincolo. Leggere "*Violazione del vincolo unq_cf*" permette al programmatore di capire istantaneamente che c'è un problema di codici fiscali duplicati, senza dover fare debug alla cieca.
2. **Manipolazione a posteriori (Enable/Disable):** Quando il vincolo viene dichiarato, il DBMS lo crea e lo attiva automaticamente. Tuttavia, conoscendone il nome, l'amministratore del database può manipolarlo in futuro senza distruggere i dati. Tramite istruzioni specifiche (es. `ALTER TABLE ... DISABLE CONSTRAINT`), un vincolo può essere momentaneamente spento. Questa operazione è comunissima durante l'importazione massiva di milioni di dati (*bulk insert*), dove spegnere i controlli accelera drasticamente la scrittura, per poi riattivarli (*ENABLE*) a importazione finita.

La motivazione tecnica principale che rende indispensabile la sintassi a livello di tabella è la gestione dei vincoli multi-colonna.

Nel nostro esempio precedente, la chiave primaria era formata da un solo attributo (*matricola*). Ma cosa accade se la progettazione logica produce una **chiave primaria composta** (es. una tabella che memorizza gli esami superati da uno studente in un determinato corso)? In SQL è illegale scrivere `PRIMARY KEY` su due righe separate, perché il DBMS tenterebbe di creare *due* chiavi primarie distinte per la stessa tabella. In questo scenario, **l'uso del vincolo di tabella diventa strutturalmente obbligatorio**, poiché permette di racchiudere più colonne sotto un'unica regola.

Esempio 3.3 (Riscrittura e Chiavi Composte). Mostriamo la sintassi applicata allo schema `studenti` e a una nuova tabella associativa `esami_sostenuti`.

```

1  -- Esempio 1: Riscrittura con vincoli di tabella nominati
2  CREATE TABLE studenti (
3      matricola CHAR(9),
4      cf        CHAR(16) NOT NULL, -- NOT NULL resta solitamente in
      -line
5      nome      VARCHAR(20) NOT NULL,
6      cognome   VARCHAR(20) NOT NULL,
7      dataN     DATE,
8
9      -- Sezione Vincoli di Tabella
10     CONSTRAINT pk_studenti PRIMARY KEY (matricola),

```



```

11      CONSTRAINT uq_studenti_cf UNIQUE (cf)
12 );
13
14 -- Esempio 2: Obbligatorietà del vincolo per PK composta
15 CREATE TABLE esame_sostenuto (
16     studente_matr CHAR(9),
17     codice_corso   CHAR(5),
18     voto           INT NOT NULL,
19
20     -- La PK unisce due attributi e deve essere definita qui
21     CONSTRAINT pk_esami PRIMARY KEY (studente_matr, codice_corso)
22 );

```

✓ Convenzione e Best Practice

Come regola pratica di sviluppo: il vincolo d'obbligatorietà (`NOT NULL`) si definisce **sempre e solo in-line** accanto all'attributo. Al contrario, i vincoli strutturali d'identità (`PRIMARY KEY`, `UNIQUE`) e quelli di navigabilità (`FOREIGN KEY`, che vedremo a breve) vengono quasi sempre traslati a fine tabella e nominati esplicitamente, per favorire l'ordine mentale e la manutenzione del codice.

3.4.5 Classificazione Concettuale dei Vincoli

Nei paragrafi precedenti abbiamo analizzato una classificazione prettamente *sintattica* (vincoli *in-line* vs *out-of-line*), che riguarda esclusivamente la forma e il posizionamento del codice all'interno dell'istruzione `CREATE TABLE`.

Esiste tuttavia una classificazione ben più importante, di natura **concettuale**, che categorizza i vincoli in base al loro "raggio d'azione", ovvero alla porzione di dati che il DBMS deve leggere per poter verificare se il vincolo è rispettato o violato. Secondo questa logica, i vincoli si dividono in quattro categorie a complessità crescente:

1. **Vincoli di Dominio (o di Colonna):** Esprimono condizioni sui valori ammissibili per un singolo attributo, valutando la singola cella isolata dal resto. Sono i vincoli più leggeri da verificare per il motore del database.
 - *Osservazione sintattica:* Questo è tipicamente l'unico tipo di vincolo che ha senso logico definire *in-line*.
 - *Esempi già incontrati:* Il vincolo di obbligatorietà (`NOT NULL`) e l'assegnazione di un valore predefinito (`DEFAULT`). Per verificare se un campo è nullo, il DBMS deve guardare solo quella specifica cella.
2. **Vincoli di n-pla (o di Riga):** Esprimono condizioni logiche che coinvolgono contemporaneamente più attributi appartenenti alla stessa riga (tupla). Per verificare il vincolo, il DBMS deve estrarre l'intera riga, ma non ha bisogno di confrontarla con le altre righe della tabella.
3. **Vincoli Intra-relazionali (o di Tabella):** Esprimono condizioni che coinvolgono più record all'interno della stessa tabella. Per validare il nuovo dato, il DBMS è costretto a scansionare o interrogare le altre righe già presenti.
 - *Esempi già incontrati:* `UNIQUE` e `PRIMARY KEY`. Per garantire che un Codice Fiscale sia univoco, non basta guardare la riga appena inserita; il database deve verificare che quel valore non sia già presente in nessun altro record della tabella.

4. **Vincoli Inter-relazionali (o di Schema):** Esprimono condizioni che legano tra loro record appartenenti a due o più tabelle distinte. Sono i vincoli più complessi e costosi computazionalmente, in quanto costringono il DBMS a "navigare" tra relazioni diverse per validare l'integrità strutturale del database.

Esempio 3.4 (Istanziamento della Tabella Esami e Classificazione dei Vincoli). Esemplichiamo la classificazione concettuale dei vincoli modellando la tabella **esami**, la quale memorizza la registrazione definitiva degli esami superati dagli studenti.

Dal punto di vista della *business logic*:

- La chiave primaria deve essere composta dalla coppia (**matr**, **corso**), garantendo che uno studente possa registrare un determinato esame una sola volta.
- Il voto deve essere unicamente compreso tra 18 e 30 (esame superato).
- La lode (valori 'S'/'N', default 'N') può essere assegnata solo se il voto è pari a 30.
- La matricola deve fare riferimento a uno studente reale.

Ecco il codice SQL DDL definitivo:

```

1 CREATE TABLE esame (
2     matr    CHAR(9),
3     corso   VARCHAR(20) NOT NULL,
4     data    DATE NOT NULL,
5     voto    INTEGER NOT NULL,
6     lode    CHAR(1) DEFAULT 'N',
7
8     -- Sezione Vincoli di Tabella
9     CONSTRAINT pk_esame PRIMARY KEY (matr, corso),
10
11     CONSTRAINT chk_voto_superato CHECK (voto BETWEEN 18 AND 30),
12
13     CONSTRAINT chk_lode_coerenza CHECK (lode IN ('S', 'N') AND (
14         lode = 'N' OR voto = 30)),
15
16     CONSTRAINT fk_esame_studente FOREIGN KEY (matr)
17         REFERENCES studente(matricola)
18 );

```

Analizziamo ogni singolo vincolo applicato alla tabella **esami**, esplicitando la categoria concettuale di appartenenza in base al suo spettro d'azione:

1. Vincoli di Dominio (o di colonna):

- **NOT NULL** (su **corso**, **data**, **voto**): Sono vincoli di dominio in-line. Impongono che le singole celle non siano vuote. Il DBMS verifica la singola cella isolata.
- **DEFAULT 'N'** (su **lode**): È una direttiva di dominio che interviene sulla singola cella in assenza di dato.
- **chk_voto_superato** (**CHECK (voto BETWEEN 18 AND 30)**): È un vincolo di dominio out-of-line. Anche se scritto in fondo, analizza **esclusivamente** i valori dell'attributo **voto**, isolato dal resto della riga.

2. Vincoli di n-pla (o di tupla/riga):

- **chk_lode_coerenza**: Questo vincolo impone che il carattere sia 'S' o 'N' e che, qualora sia 'S', il voto sia tassativamente 30. Dal momento che l'espressione logica mette in relazione **due attributi distinti dello stesso record** (**lode** e **voto**), si tratta di un puro vincolo di n-pla. Il DBMS deve leggere l'intera riga per validarlo.

3. Vincoli Intra-relazionali (o di Tabella):

- **pk_esame** (PRIMARY KEY (matr, corso)): Definisce l'identificatore. Per garantire che la coppia matricola-corso sia unica, il DBMS non può limitarsi a guardare la riga corrente, ma deve scansionare l'intera tabella **esame** per escludere duplicati. Agisce quindi a livello di **più record della stessa tabella**.

4. Vincoli Inter-relazionali (o di Schema):

- **fk_esame_studente** (FOREIGN KEY): È il vincolo che garantisce l'integrità referenziale. Stabilisce un legame tra la tabella corrente (**esami**) e una tabella esterna (**studente**). Il database, per accettare la riga, è costretto a saltare fuori dalla tabella corrente e verificare la presenza della matricola nello schema dello studente.

3.4.6 Integrità Referenziale e Chiavi Esterne (FOREIGN KEY)

Il vincolo di **Chiave Esterna (FOREIGN KEY)** è il più importante tra i vincoli **inter-relazionali** (o di schema). Esso stabilisce una relazione di dipendenza tra due tabelle, imponendo che i valori presenti in un set di attributi della tabella corrente (*tabella figlia* o *referenziante*) debbano obbligatoriamente esistere all'interno del set di attributi che costituisce la chiave primaria (o un'altra chiave univoca) di un'altra tabella (*tabella padre* o *referenziata*).

Matematicamente, garantisce l'**Integrità Referenziale**: non possono esistere "puntatori orfani" nel database.

Come abbiamo visto nel paragrafo precedente, la sintassi standard a livello di tabella è la seguente:

```
1 CONSTRAINT <nome_fk> FOREIGN KEY (colonna_locale_1, ...)
2 REFERENCES <tabella_padre> (colonna_padre_1, ...)
```

Ordine Posizionale degli Attributi Referenziati

Nel caso in cui la relazione coinvolga chiavi composte da più attributi (es. una tabella figlia che punta alla nostra tabella **esami**, la cui Primary Key è la coppia **matr, corso**), l'**ordine** in cui le colonne vengono dichiarate all'interno delle parentesi tonde è tassativo.

L'associazione effettuata dal motore SQL **non avviene per nome, ma è strettamente posizionale** (uno-a-uno): il primo attributo elencato in **FOREIGN KEY** viene mappato sul primo attributo elencato in **REFERENCES**, il secondo sul secondo, e così via. Un'inversione accidentale dell'ordine genererà un errore di compilazione (se i domini non corrispondono) o, nel peggiore dei casi, una drammatica corruzione semantica dei dati referenziati.

Il problema delle modifiche sui dati referenziati

Il vero fulcro ingegneristico della chiave esterna emerge quando si tenta di alterare lo stato della tabella padre. Supponiamo che nel nostro database esista uno studente con matricola '12345' e che nella tabella **esami** vi siano tre record associati a tale matricola.

Cosa accade se un operatore tenta di **cancellare** lo studente '12345' dalla tabella **studenti**, o di **modificarne** la matricola? Se il DBMS permettesse questa operazione

alla leggera, i tre record della tabella **esami** punterebbero a una matricola inesistente, violando l'integrità referenziale.

Per risolvere questo problema, lo Standard SQL impone al progettista di specificare esplicitamente una **politica di reazione** sia per la cancellazione (**ON DELETE**) sia per l'aggiornamento (**ON UPDATE**).

Il progettista può scegliere tra quattro comportamenti distinti, da dichiarare in coda al vincolo di chiave esterna:

1. **RESTRICT / NO ACTION (Comportamento di Default):** Il DBMS blocca l'operazione alla radice. Se si tenta di cancellare lo studente padre, il database lancia un'eccezione di violazione di vincolo e rifiuta la query, costringendo l'utente a cancellare manualmente prima tutti gli esami correlati.
 - Sintassi: **ON DELETE RESTRICT**
2. **CASCADE (Cancellazione/Modifica a cascata):** L'operazione eseguita sul padre si ripercuote automaticamente sui figli. Se cancello lo studente, il DBMS cancella autonomamente tutte le righe dei suoi esami. Se modifico la matricola del padre, il DBMS aggiorna la matricola in tutti gli esami figli.
 - Sintassi: **ON DELETE CASCADE**
3. **SET NULL (Annullamento del puntatore):** Se il padre viene eliminato o modificato, i record figli sopravvivono, ma il valore della loro chiave esterna viene impostato automaticamente a **NULL**.
 - Sintassi: **ON DELETE SET NULL**
 - *Vincolo strutturale:* Questa politica è applicabile **solo se** le colonne della chiave esterna nella tabella figlia ammettono il valore **NULL** (ovvero non sono marcate come **NOT NULL** o non fanno parte della Primary Key).
4. **SET DEFAULT (Ripristino del valore predefinito):** Simile a **SET NULL**, ma i campi della chiave esterna nei record figli assumono il valore specificato nella clausola **DEFAULT** di quella colonna.
 - Sintassi: **ON DELETE SET DEFAULT**

Euristiche di scelta delle politiche

La selezione della politica referenziale dipende esclusivamente dal significato semantico della relazione:

- Si usa **CASCADE** quando la tabella figlia descrive entità deboli che non hanno senso di esistere senza il padre (es. le righe di un ordine d'acquisto: se cancello l'ordine, devo distruggere le sue righe).
- Si usa **RESTRICT** quando l'eliminazione del padre sarebbe un errore operativo grave (es. non devo poter cancellare un corso di laurea se ci sono ancora studenti iscritti ad esso).
- Si usa **SET NULL** quando l'entità figlia ha vita propria ma perde il legame con il padre (es. se una scuderia di Formula 1 fallisce e si cancella la scuderia, i piloti rimangono nel database ma con la colonna **id_scuderia** impostata a **NULL**).

Dichiarazione Multipla e Comportamenti Misti

È prassi comune assegnare comportamenti logici diversi per la cancellazione e per l'aggiornamento. Sintatticamente, le due clausole si scrivono in sequenza all'interno

- ✓ della dichiarazione del vincolo.

Esempio di combinazione (Best Practice industriale):

```
1 CONSTRAINT fk_esami_studenti FOREIGN KEY (matr)
2 REFERENCES studenti(matricola)
3 ON DELETE RESTRICT      -- Protegge dalla perdita
   accidentale dei dati
4 ON UPDATE CASCADE       -- Garantisce flessibilita'
   operativa sui codici
```

In questo specifico (e comunissimo) scenario, il DBMS impedirà la cancellazione di uno studente che ha già sostenuto esami, ma permetterà alla segreteria di correggerne o aggiornarne la matricola, propagando istantaneamente il nuovo codice in tutte le righe della tabella **esami**.

✓ Design Pattern: Il Record Fittizio (Dummy Record)

Durante la modellazione fisica, la gestione dei valori **NULL** nelle chiavi esterne può generare complessità in fase di interrogazione dell'applicativo (necessità di *Outer Join* e gestione delle incognite).

Per ovviare a questo problema, i progettisti adottano spesso il pattern del **Record Fittizio**. La tecnica consiste nell'inserire nella tabella padre una tupla convenzionale (es. con Primary Key 0 o -1 e descrizione "Non Assegnato"). Nella tabella figlia, la colonna dedicata alla chiave esterna viene dichiarata **NOT NULL** (impedendo l'incognita) e le viene assegnato come **DEFAULT** l'identificatore del record fittizio.

Vantaggi architettonici:

- **Integrità totale:** Ogni riga figlia punta sempre a un record padre esistente, eliminando il concetto di "puntatore vuoto".
- **Sinergia con le politiche referenziali:** Questo pattern rende implementabile in modo logico la direttiva **ON DELETE SET DEFAULT**. Se un'entità padre viene eliminata, i record figli ad essa associati non assumono il valore **NULL**, ma vengono automaticamente riassegnati al record "Sconosciuto" (valore di default), preservando la continuità strutturale dei dati.

⚠ In alcuni contesti didattici, con **Vincolo di Integrità Referenziale** si intende che la FK nella tabella figlia debba **sempre** puntare ad una tupla nella tabella padre, eventualmente un dummy record (ad esempio, nel caso in cui la FK sia **NULL**).

SQL e Algebra Relazionale

In questo capitolo studieremo in parallelo ulteriori elementi del linguaggio SQL e l'algebra relazionale; non si tratta di un mero esercizio didattico, ma la chiave di volta per comprendere la reale architettura di calcolo di un Database Management System (DBMS).

Esiste una dicotomia fondamentale tra i due strumenti, che si riflette nella natura stessa dei linguaggi:

- **L'Algebra Relazionale è procedurale (o costruttiva):** Definisce rigorosamente la sequenza dei passi (le operazioni elementari come selezioni, proiezioni e join) necessari per manipolare le relazioni e ottenere il risultato.
- **SQL è un linguaggio di alto livello dichiarativo (o descrittivo):** Il programmatore si limita a definire le proprietà dell'insieme dei dati che desidera ottenere (*cosa*), ignorando del tutto la strategia fisica per reperirli (*come*). Un'unica istruzione **SELECT** agisce come un costrutto sintattico che incapsula molteplici operazioni algebriche.

Comprendere l'algebra relazionale significa avere la visibilità strutturale su ciò che il DBMS esegue "sotto il cofano". Quando il sistema riceve una **SELECT**, innesca un processo di traduzione e ottimizzazione totalmente trasparente all'utente:

1. **Parsing e Traduzione:** L'istruzione SQL viene analizzata e tradotta in un albero sintattico di espressioni di algebra relazionale.
2. **Ottimizzazione Algebrica (Query Optimizer):** È il motore matematico del DBMS. L'ottimizzatore sfrutta le proprietà di equivalenza degli operatori relazionali (es. commutatività, associatività e distributività) per trasformare l'espressione iniziale in un'espressione logicamente equivalente ma con un costo computazionale minimo (ad esempio, eseguendo le selezioni per ridurre la cardinalità degli insiemi prima di effettuare un prodotto cartesiano).
3. **Piano di Esecuzione (Execution Plan):** L'espressione algebrica ottimizzata viene mappata in operazioni fisiche di accesso alla memoria (o al disco) e infine eseguita.

Q Limiti di scopo e Amministrazione della Base di Dati

I DBMS relazionali mettono a disposizione dei progettisti istruzioni di basso livello (dette *Hint*) per manipolare e forzare manualmente i piani di esecuzione, esautorando di fatto il Query Optimizer dalle sue scelte automatiche.

Tuttavia, queste operazioni di *Query Tuning* fisico appartengono a un dominio avanzato (tipico dell'amministrazione di database o di corsi di livello magistrale). L'obiettivo di questo documento si limita al livello logico-dichiarativo: ci concentreremo sulla **correttezza semantica** delle interrogazioni scritte, delegando completamente l'efficienza procedurale e la gestione algoritmica al motore del DBMS.

Prima di introdurre le operazioni algebriche, è imperativo distinguere l'aspetto intensionale (la struttura) dall'aspetto estensionale (i dati) di una base di dati:

- **Schema Relazionale (Intensione):** Rappresenta la struttura statica della tabella. Definisce il nome della relazione e l'elenco dei suoi attributi con i rispettivi domini. Esempio: *STUDENTE(matricola, nome, cognome, dataN, città)*.

- **Relazione (Estensione):** Corrisponde all'istanza dello schema in un dato istante di tempo, ovvero alle righe della tabella. Dal momento che il modello si fonda sulla teoria degli insiemi, una relazione è rigorosamente un **insieme di tuple**.

matricola	nome	cognome	dataN	citta	lavoro
10001	Mario	Rossi	1999-05-14	Roma	NULL
10002	Luigi	Verdi	2000-11-22	Milano	Barista
10003	Giulia	Rossi	2001-02-10	Roma	NULL
10004	Anna	Bianchi	1999-08-30	Napoli	Developer
10005	Mario	Neri	2000-01-15	Milano	NULL

Tabella 4.1: Istanza aggiornata della relazione **Studente**, con l'attributo parziale **Lavoro**.

Osservazione 4.1 (Proprietà Insiemistiche della Relazione). Ricordiamo che, trattandosi di un insieme matematico, per definizione:

1. **Non ammette duplicati:** Non possono esistere due tuple identiche in ogni loro attributo.
2. **L'ordine è irrilevante:** Cambiando l'ordine fisico delle righe, l'insieme rimane matematicamente lo stesso.

4.1 Proiezioni

La proiezione è un operatore **unario** (agisce su una singola relazione) che effettua una "decomposizione verticale". In termini pratici, estrae una sottotabella conservando solo gli attributi specificati e scartando gli altri.

Sia $R(X)$ uno schema di relazione, e sia $Y \subseteq X$ un sottoinsieme dei suoi attributi. La proiezione della relazione r sull'insieme di attributi Y è una relazione sullo schema $R(Y)$ che si denota con $\pi_Y(r)$.

- **Schema risultante:** Lo schema della relazione proiettata contiene meno attributi (o al massimo gli stessi) della relazione di partenza. A differenza dell'ordine delle tuple (irrilevante), l'**ordine degli attributi** specificato nell'operatore definisce la struttura della nuova relazione (es. $\pi_{nome,cognome}(r)$ produce uno schema diverso da $\pi_{cognome,nome}(r)$).
- **Riduzione della cardinalità (De-duplicazione):** Poiché il risultato dell'operazione deve essere ancora una relazione (un insieme), il numero di tuple della proiezione può essere inferiore al numero di tuple della relazione originale. Se la rimozione delle colonne genera tuple identiche, l'operatore matematico "collassa" i duplicati in un unico elemento.
- **Composizione:** Essendo chiusa rispetto all'algebra, è possibile applicare proiezioni in cascata. Risulta banale dimostrare che se $Z \subseteq Y \subseteq X$, allora $\pi_Z(\pi_Y(r)) = \pi_Z(r)$.

Esempio 4.1. Applichiamo l'operatore di proiezione all'istanza della relazione **Studente** precedentemente definita in tabella 4.1:

$\pi_{nome,cognome}(\text{studente})$	$\pi_{cognome,nome}(\text{studente})$	$\pi_{cognome}(\text{studente})$
nome cognome	cognome nome	cognome
Mario Rossi	Rossi Mario	Rossi
Luigi Verdi	Verdi Luigi	Verdi
Giulia Rossi	Rossi Giulia	Bianchi
Anna Bianchi	Bianchi Anna	Neri
Mario Neri	Neri Mario	

Tabella 4.2: Confronto delle proiezioni: l'ultima operazione evidenzia la riduzione della cardinalità (da 5 a 4 tuple) dovuta alla de-duplicazione insiemistica.

4.1.1 La Proiezione in SQL: Il comando SELECT

Il linguaggio SQL implementa l'operatore di proiezione tramite la clausola **SELECT**. Il risultato di un'interrogazione non viene salvato fisicamente sul disco (a meno di comandi DDL espliciti), ma costituisce una **tabella virtuale** (o *Result Set*) che risiede temporaneamente nella memoria di lavoro.

Sintassi generale della proiezione SQL:

```

1 SELECT [ALL | DISTINCT] <elenco_attributi>
2 FROM <nome_tabella>;

```

Esiste una divergenza critica tra l'operatore matematico π e la clausola **SELECT**:

- **SELECT ALL (Comportamento di default):** SQL, per ottimizzare le prestazioni computazionali, tratta le tabelle come *multinsiemi* (bag). Se calcoliamo **SELECT cognome FROM studente**, e nel database esistono dieci studenti di nome "Rossi", la tabella virtuale risultante conterrà dieci righe identiche. Questo viola la definizione di insieme.
- **SELECT DISTINCT:** Per forzare il DBMS a comportarsi in modo rigoroso secondo le regole dell'algebra relazionale, è necessario utilizzare la keyword **DISTINCT**. Questa impone al motore di ordinare i risultati ed eliminare i duplicati, restituendo un insieme matematico puro (a fronte di un maggiore costo di calcolo).

4.2 Selezioni

La selezione è, come la proiezione, un operatore **unario** che estrae un sottoinsieme delle tuple che soddisfano una determinata condizione logica, lasciando inalterata la struttura della tabella (a differenza delle proiezioni).

Sia $R(X)$ uno schema di relazione e sia p un predicato (condizione logica) definito sugli attributi di X . La selezione della relazione r rispetto al predicato p è una relazione sullo stesso schema $R(X)$ che si denota con $\sigma_p(r)$.

- **Schema risultante:** Lo schema della relazione selezionata è identico allo schema della relazione di partenza. Il numero, il nome e l'ordine degli attributi non subiscono variazioni.

- **Riduzione della cardinalità:** Il risultato è un sottoinsieme insiemistico della relazione originale. Il numero di tuple prodotte è minore o uguale al numero di tuple di partenza.
- **Composizione:** L'applicazione di selezioni in cascata equivale a una singola selezione il cui predicato è la congiunzione logica (AND) dei singoli predicati. Risulta banale dimostrare che $\sigma_{p_1}(\sigma_{p_2}(r)) = \sigma_{p_1 \wedge p_2}(r)$.

Esempio 4.2. Applichiamo l'operatore di selezione all'istanza della relazione **studente** (Tabella 4.1) per estrarre le sole tuple relative agli studenti residenti a Roma:

$$\sigma_{citta='Roma'}(\text{studente})$$

matricola	nome	cognome	dataN	citta	lavoro
10001	Mario	Rossi	1999-05-14	Roma	NULL
10003	Giulia	Rossi	2001-02-10	Roma	NULL

Tabella 4.3: Risultato della selezione: lo schema resta inalterato (6 attributi), ma la cardinalità si riduce alle sole tuple che rendono vero il predicato.

4.2.1 La Selezione in SQL: La clausola WHERE

In SQL, l'operazione logica di selezione si implementa aggiungendo la clausola **WHERE** al blocco di interrogazione. Il predicato dell'algebra relazionale viene letteralmente tradotto in un'espressione condizionale.

Sintassi generale della selezione SQL:

```

1 SELECT *
2 FROM <nome_tabella>
3 WHERE <condizione>;

```

Nelle interrogazioni SQL, l'asterisco (*) agisce come un carattere jolly (wildcard) universale che indica **"tutti gli attributi"**.

Dal punto di vista dell'algebra relazionale, la clausola **SELECT** implementa l'operatore di proiezione (π). Tuttavia, per limiti sintattici storici del linguaggio SQL, un'interrogazione deve *obbligatoriamente* iniziare con il comando **SELECT**, anche qualora il progettista desideri eseguire unicamente un'estrazione orizzontale (selezione σ), mantenendo intatta la struttura della tabella.

L'asterisco fornisce la scappatoia formale a questo vincolo sintattico: comunica al motore del DBMS di **non effettuare alcuna operazione di decomposizione verticale**, restituendo la tupla nella sua interezza.

- **Proiezione esplicita:** **SELECT nome, cognome** equivale a $\pi_{nome,cognome}(r)$
- **Nessuna proiezione:** **SELECT *** preserva lo schema $R(X)$ originale inalterato.

Pertanto, l'espressione algebrica composta esclusivamente da una selezione

$\sigma_{citta='Roma'}(\text{studente})$ si traduce in SQL in:

```

1 SELECT * FROM studente
2 WHERE citta = 'Roma';

```


⚠ L'anti-pattern del SELECT *

Sebbene l'uso di `SELECT *` sia estremamente comodo durante l'ispezione manuale dei dati, il suo impiego all'interno del codice sorgente di un'applicazione software (Backend) è universalmente considerato un **anti-pattern** per due ragioni architetturali:

1. **Spreco computazionale (I/O e Rete):** Trasferire colonne non necessarie all'applicativo consuma inutilmente larghezza di banda e memoria RAM.
2. **Fragilità strutturale (Breaking Changes):** Se in futuro lo schema del database dovesse evolvere (`ALTER TABLE`) con l'aggiunta di nuove colonne pesanti (es. dati binari) o sensibili, la vecchia query inizierebbe silenziosamente a scaricarle, potendo causare saturazione della memoria o vulnerabilità di sicurezza.

La prassi ingegneristica standard impone di **elencare sempre esplicitamente** gli attributi desiderati dopo la clausola `SELECT`, persino nel caso in cui coincidano con l'intero schema relazionale in quel preciso momento.

4.2.2 Attributi Parziali e Logica Ternaria

Un aspetto teorico e pratico cruciale riguarda la valutazione dei predicati in presenza di attributi parziali (non obbligatori). Si assume per definizione che il valore speciale **NULL** sia sempre incluso nel dominio di qualsiasi attributo, a meno che la sua presenza non sia esplicitamente vietata da un vincolo strutturale (es. `NOT NULL` o `PRIMARY KEY`).

Poiché `NULL` rappresenta l'incognita ("valore sconosciuto" o "inapplicabile"), **qualsiasi tipo di predicato o operazione** (non solo i normali operatori di confronto relazionale come `=`, `>`, `<`, `<>`, ma anche operatori avanzati come `LIKE`, `IN` o `BETWEEN`) fallisce nel restituire un risultato certo. Valutare matematicamente un'incognita all'interno di una condizione non restituisce necessariamente `TRUE` o `FALSE`, ma introduce un terzo stato logico: **UNKNOWN**.

Nelle espressioni booleane complesse, l'algebra relazionale adotta dunque una logica a tre valori (ternaria). Il principio per risolvere le tabelle di verità senza memorizzarle è puramente intuitivo: **il risultato globale è TRUE o FALSE solo se l'incognita non è più determinante per l'esito; altrimenti resta UNKNOWN**.

- `TRUE AND UNKNOWN = UNKNOWN` (L'esito dipende dall'incognita).
- `FALSE AND UNKNOWN = FALSE` (Un solo `FALSE` in un `AND` è sufficiente per falsificare tutto, l'incognita è irrilevante).
- `TRUE OR UNKNOWN = TRUE` (Un `TRUE` in un `OR` è sufficiente per verificare tutto).
- `FALSE OR UNKNOWN = UNKNOWN` (L'esito dipende dall'incognita).

⚠ Criterio di Ammissione della Tupla

Affinché l'operatore di selezione σ includa una tupla nel risultato finale, il predicato complessivo deve valutarsi **strettamente a TRUE**. Qualsiasi tupla che restituisca `FALSE` o `UNKNOWN` viene inesorabilmente scartata.

Operatori per l'identificazione delle incognite Per poter filtrare deliberatamente le tuple in base all'assenza di dati, SQL introduce due operatori dedicati che restituiscono sempre `TRUE` o `FALSE`, bypassando lo stato `UNKNOWN`:

- **IS NULL**: Valuta a TRUE se la cella è priva di valore.
- **IS NOT NULL**: Valuta a TRUE se la cella contiene un valore definito e valido.

Esempio 4.3 (Filtrare tramite operatori dedicati: studenti non lavoratori). Per selezionare gli studenti che non hanno un impiego registrato, è necessario intercettare il valore NULL usando l'operatore specifico, non l'uguaglianza.

$$\sigma_{\text{lavoro IS NULL}}(\text{studente})$$

```
1 SELECT * FROM studente WHERE lavoro IS NULL;
```

matricola	nome	cognome	dataN	citta	lavoro
10001	Mario	Rossi	1999-05-14	Roma	NULL
10003	Giulia	Rossi	2001-02-10	Roma	NULL
10005	Mario	Neri	2000-01-15	Milano	NULL

Esempio 4.4 (Insidia degli operatori standard con il valore NULL). Si voglia estrarre l'elenco degli studenti il cui lavoro è **diverso** da 'Developer'.

$$\sigma_{\text{lavoro} \neq \text{'Developer'}}(\text{studente})$$

```
1 SELECT * FROM studente WHERE lavoro <> 'Developer';
```

È un errore logico comune aspettarsi che gli studenti disoccupati (con lavoro a NULL) vengano inclusi. In realtà, valutare $\text{NULL} \neq \text{'Developer'}$ restituisce UNKNOWN. Di conseguenza, la selezione ammette solo chi ha un lavoro esplicitato diverso da quello cercato, scartando i NULL.

matricola	nome	cognome	dataN	citta	lavoro
10002	Luigi	Verdi	2000-11-22	Milano	Barista

4.3 Composizione di Proiezioni e Selezioni

Nell'algebra relazionale, le interrogazioni complesse si costruiscono componendo gli operatori di base. Poiché il risultato di un'operazione relazionale è sempre a sua volta una relazione (proprietà di chiusura algebrica), gli operatori possono essere annidati e applicati in cascata.

L'espressione più comune consiste nell'estrazione di specifiche colonne da un sottoinsieme filtrato di tuple. Matematicamente, questo si modella applicando prima una selezione e successivamente una proiezione sul risultato:

$$\pi_Y(\sigma_p(r))$$

dove r è la relazione di partenza, p è il predicato logico di selezione e Y è l'insieme degli attributi da proiettare.

A differenza dell'algebra classica o della composizione tra operatori identici (es. due selezioni consecutive commutano sempre), **la proiezione e la selezione non commutano in generale**:

$$\pi_Y(\sigma_p(r)) \neq \sigma_p(\pi_Y(r))$$

Questa disuguaglianza diventa un vero e proprio errore di valutazione se la condizione logica p coinvolge attributi della relazione originale che *non* sono inclusi nell'insieme proiettato Y . Se la proiezione venisse eseguita per prima, scarterebbe prematuramente le colonne necessarie per valutare la condizione della selezione successiva.

Esempio 4.5 (Controesempio sulla Non-Commutatività). Consideriamo l'istanza della relazione **studente** (Tabella 4.1) e supponiamo di voler estrarre solo i **nomi** degli studenti residenti a **Roma**.

1. Composizione Corretta: $\pi_{nome}(\sigma_{citta='Roma'}(studente))$

- L'operatore più interno (σ) filtra le tuple tenendo solo quelle che soddisfano **Citta** = 'Roma'.
- L'operatore esterno (π) riceve la relazione filtrata e scarta tutte le colonne eccetto il nome.
- **Risultato:** Una relazione formalmente corretta contenente le tuple "Mario" e "Giulia".

2. Composizione Errata (Commutazione): $\sigma_{citta='Roma'}(\pi_{nome}(studente))$

- L'operatore più interno (π) scarta subito tutte le colonne eccetto **Nome**. Lo schema risultante diviene $R_{tmp}(nome)$.
- L'operatore esterno (σ) tenta di valutare la condizione **citta** = 'Roma' sulle tuple di R_{tmp} , ma l'attributo **citta** **non esiste più** nello schema corrente.
- **Risultato:** L'espressione è **matematicamente indefinita (mal formata)**. Poiché l'attributo **citta** non appartiene allo schema di R_{tmp} , il predicato non può essere logicamente valutato (non restituisce "Falso" generando un insieme vuoto, ma costituisce una violazione del dominio di definizione). In SQL, questa interrogazione non restituisce un *result set* vuoto, ma genera un errore semantico a tempo di compilazione.

4.3.1 La Composizione in SQL e l'Ordine Logico di Esecuzione

Il linguaggio SQL traduce la composizione $\pi_Y(\sigma_p(r))$ avvalendosi simultaneamente delle clausole **SELECT** (per la proiezione) e **WHERE** (per la selezione):

```
1 SELECT nome
2 FROM studente
3 WHERE citta = 'Roma';
```

Per evitare a livello architetturale i paradossi della non-commutatività appena dimostrati, lo standard SQL impone al Query Optimizer un rigoroso **ordine logico di valutazione**, che differisce dall'ordine di scrittura del codice sorgente (dall'alto verso il basso).

Il DBMS analizza l'interrogazione applicando le operazioni esattamente nella sequenza insiemistica sicura:

1. **FROM:** Identifica la relazione (o le relazioni) di partenza.
2. **WHERE:** Applica la condizione di selezione (σ), filtrando orizzontalmente le righe valide.
3. **SELECT:** Applica la proiezione (π) sull'insieme filtrato, scartando verticalmente le colonne escluse.

Di conseguenza, nella clausola **WHERE** è sempre possibile e legittimo fare riferimento ad attributi della tabella che non compaiono nella **SELECT**, poiché la valutazione del filtro avviene *prima* del taglio delle colonne.

4.4 Qualificazione degli Attributi

Finora abbiamo utilizzato una sintassi semplificata per indicare le colonne (es. **SELECT nome**). Tuttavia, la teoria dei database relazionali impone che il riferimento a un attributo debba essere assoluto e privo di ambiguità.

La notazione estesa, o **qualificazione**, si esprime tramite la sintassi (vedremo gli alias nel paragrafo a seguire):

nome_tabella.nome_attributo oppure alias_tabella.nome_attributo

Osservazione 4.2 (Risoluzione delle Ambiguità). Quando un'interrogazione agisce su una singola tabella, il DBMS risolve implicitamente l'appartenenza dell'attributo. Tuttavia, quando si lavora su prodotti cartesiani o *Join* tra più tabelle (es. **studente** e **professore**), è matematicamente frequente che due tabelle possiedano attributi omonimi (es. **nome** e **cognome**).

In questi scenari, omettere la qualificazione genera un errore semantico bloccante (*Ambiguous column name*). Pertanto, è considerata una **best practice ingegneristica e di stile** adottare sempre la sintassi estesa **tabella.attributo** (spesso in sinergia con un alias di tabella) non appena l'interrogazione coinvolge più di una relazione.

4.5 Ridenominazione

La ridenominazione è un operatore **unario** (agisce su una singola relazione) atipico: a differenza di proiezione e selezione, non filtra né estrae dati, ma agisce esclusivamente sull'intensione (lo schema), lasciando inalterata l'estensione (le tuple). In termini pratici, modifica il nome della tabella o dei suoi attributi.

Sia $R(X)$ uno schema di relazione. L'operatore di ridenominazione si denota tipicamente con la lettera greca rho (ρ). Può assumere tre forme:

- **Ridenominazione di attributi:** $\rho_{nuovo \leftarrow vecchio}(r)$ altera il nome di specifiche colonne.
- **Ridenominazione di relazione:** $\rho_s(r)$ assegna il nuovo nome s all'intera relazione.
- **Ridenominazione completa (relazione e attributi):** $\rho_{s(a_1, a_2, \dots, a_n)}(r)$ assegna il nuovo nome s all'intera relazione e, simultaneamente, rinomina **tutti** i suoi attributi in $a_1, a_2, \dots, a_n$. Affinché l'espressione sia matematicamente ben formata, il numero dei nuovi attributi forniti tra parentesi deve corrispondere esattamente al grado (numero di colonne) della relazione di partenza. L'assegnazione avviene per rigoroso ordine posizionale (da sinistra verso destra).

Lo schema della relazione prodotta possiede lo stesso grado (numero di attributi) e gli stessi identici domini della relazione di partenza, ma identificatori testuali differenti; anche il numero e il contenuto delle tuple rimane del tutto invariato.

Sebbene ad una prima occhiata non sembri un operatore di grande utilità, vedremo presto che è fondamentale per la risoluzione di ambiguità.

4.5.1 La Ridenominazione in SQL: L'operatore AS

Il linguaggio SQL implementa l'operatore algebrico ρ attraverso la parola chiave **AS** (spesso definita *Alias*).

Coerentemente con la natura duale dell'operatore matematico (che agisce su attributi o su relazioni), anche in SQL l'alias può essere impiegato in due contesti architetturali distinti:

Ridenominazione degli Attributi (nella clausola SELECT) Applicato all'elenco delle colonne, l'operatore modifica l'intestazione della tabella virtuale (Result Set) prodotta in output. Risulta indispensabile quando si estraggono dati derivati da espressioni matematiche o funzioni, che altrimenti verrebbero restituiti dal motore del database privi di un nome di colonna definito (o con nomi generati automaticamente come ?column?).

Sintassi per attributi:

```
1 SELECT matricola AS id_studente, cognome
2 FROM studente;
```

Ridenominazione delle Relazioni (nella clausola FROM) Applicato alle tabelle di origine, assegna un nome temporaneo all'intera relazione per la durata dell'interrogazione. Questo meccanismo, unito alla notazione puntata, è l'unico modo per risolvere le ambiguità quando si interrogano simultaneamente più tabelle che presentano attributi omonimi, o quando si incrocia una tabella con sé stessa.

Sintassi per relazioni (con notazione puntata):

```
1 SELECT s.nome, s.cognome
2 FROM studente AS s
3 WHERE s.città = 'Roma';
```

4.6 Operazioni Insiemistiche

Essendo il modello relazionale fondato sulla teoria degli insiemi, è possibile combinare due relazioni utilizzando i classici operatori insiemistici: **Unione** ($\cup$), **Intersezione** ($\cap$) e **Differenza** ($-$).

Tuttavia, diversamente dalla matematica pura dove è possibile unire insiemi eterogenei (es. un insieme di mele con un insieme di numeri), nell'algebra relazionale le tuple elaborate devono avere una struttura coerente.

⚠ Condizione di Compatibilità degli Schemi

Le operazioni insiemistiche tra due relazioni r_1 e r_2 sono definite **se e solo se** i loro schemi sono compatibili. Due schemi sono compatibili quando:

1. Hanno lo stesso numero di attributi (stesso grado).
2. Gli attributi corrispondenti (letti da sinistra verso destra) appartengono allo stesso dominio semantico.

Se questa condizione è rispettata, lo schema della relazione risultante coinciderà tipicamente con lo schema del primo operando (r_1).

Date due relazioni r_1 e r_2 compatibili:

- **Unione** ($r_1 \cup r_2$): Costruisce una relazione contenente tutte le tuple che appartengono a r_1 , a r_2 , o a entrambe. Essendo un insieme, le tuple presenti in entrambe le relazioni compaiono una sola volta nel risultato.
- **Intersezione** ($r_1 \cap r_2$): Costruisce una relazione contenente esclusivamente le tuple che appartengono contemporaneamente sia a r_1 che a r_2 .

- **Differenza** ($r_1 - r_2$): Costruisce una relazione contenente le tuple che appartengono a r_1 ma **non** appartengono a r_2 .

Per comprendere l'importanza di questi operatori, consideriamo il seguente schema relazionale che modella gli incontri di un campionato di calcio:

$$PARTITE(data, stadio, squadra1, squadra2, goals1, goals2)$$

Nota: Convenzionalmente, **squadra1** gioca in casa e **goals1** rappresenta le sue reti.

Esempio 4.6 (L'Unione: squadre vincenti allo stadio Maradona). Vogliamo ricavare l'elenco di tutte le squadre che hanno vinto almeno una partita allo stadio 'Maradona'.

Il problema logico risiede nel fatto che una squadra può aver vinto giocando in casa (**squadra1**) oppure in trasferta (**squadra2**). Non potendo estrarre due colonne diverse contemporaneamente in una singola proiezione lineare, l'unione risulta essenziale:

1. Ricaviamo chi ha vinto in casa:

$$v_{casa} = \pi_{squadra1}(\sigma_{stadio='Maradona' \wedge goals1 > goals2}(partite))$$

2. Ricaviamo chi ha vinto in trasferta:

$$v_{trasferta} = \pi_{squadra2}(\sigma_{stadio='Maradona' \wedge goals2 > goals1}(partite))$$

3. Uniamo i risultati (gli schemi sono compatibili, essendo entrambi domini di squadre):

$$risultato = v_{casa} \cup v_{trasferta}$$

Esempio 4.7 (La Differenza e la logica del "Mai"). Vogliamo ricavare l'elenco delle squadre che **non hanno mai vinto** allo stadio 'Maradona'.

Questa interrogazione non può essere risolta con una semplice selezione negata. Questo limite logico rende l'operatore di differenza insostituibile ogni volta che un requisito contiene parole come "mai" o "nessuno". L'approccio corretto è sempre per sottrazione: **Tutti - Chi ha l'attributo negato**.

1. Troviamo **tutte** le squadre del campionato:

$$tutte = \pi_{squadra1}(partite) \cup \pi_{squadra2}(partite)$$

2. Troviamo le squadre che **hanno vinto almeno una volta** al Maradona (espressione *risultato* dell'esempio precedente, chiamiamola *vincitriciM*). 3. Applichiamo la differenza:

$$maiVintoM = tutte - vincitriciM$$

Esempio 4.8 (Trappola Logica: Squadre che non hanno mai vinto in casa). Un errore classicamente commesso in fase di modellazione è tentare di risolvere i problemi di negazione universale ("mai") utilizzando i normali operatori di confronto.

Soluzione Sbagliata: Si potrebbe essere tentati di scrivere:

$$errata = \pi_{squadra1}(\sigma_{goals1 \leq goals2}(partite))$$

Perché è sbagliata? Questa espressione restituisce le squadre che hanno perso o pareggiato *almeno una partita* in casa. Tuttavia, se una squadra ha perso una partita in casa (entrando così nel risultato) ma ne ha vinta un'altra, il requisito "non ha mai vinto" viene violato.

Soluzione Corretta (tramite Differenza): 1. Troviamo tutte le squadre che hanno giocato in casa: $t_{casa} = \pi_{squadra1}(partite)$ 2. Troviamo le squadre che hanno vinto almeno una volta in casa: $v_{casa} = \pi_{squadra1}(\sigma_{goals1 > goals2}(partite))$ 3. Sottraiamo:

$$risultato = t_{casa} - v_{casa}$$

4.6.1 Le Operazioni Insiemistiche in SQL

Il linguaggio SQL implementa le operazioni insiemistiche dell'algebra relazionale attraverso i costrutti **UNION** (Unione), **INTERSECT** (Intersezione) ed **EXCEPT** (Differenza). Alcuni dialetti commerciali storici (come Oracle) utilizzano la keyword **MINUS** in luogo di **EXCEPT**.

Affinché l'interrogazione possa essere compilata ed eseguita dal DBMS senza errori semantici, i due blocchi **SELECT** coinvolti devono rigorosamente rispettare la condizione di compatibilità degli schemi vista in precedenza: stesso numero di colonne estratte e tipi di dato compatibili, valutati in rigoroso ordine posizionale.

Sintassi generale delle operazioni insiemistiche:

```

1 SELECT <elenco_attributi_A> FROM <tabella_A>
2 [ UNION | INTERSECT | EXCEPT ]
3 SELECT <elenco_attributi_B> FROM <tabella_B>;

```

Nei paragrafi precedenti abbiamo dimostrato come il comando **SELECT** base non applichi la de-duplicazione, comportandosi di default come un multinsieme (**ALL**). Con gli operatori insiemistici, lo standard SQL capovolge questa logica:

- **Comportamento di default (Puro Insieme):** L'uso di **UNION**, **INTERSECT** o **EXCEPT** forza implicitamente il motore del database a comportarsi secondo la matematica pura. Il DBMS ordinerà i risultati intermedi ed **eliminerà tutti i duplicati** (**DISTINCT** implicito).
- **Comportamento Multinsieme (ALL):** Qualora il programmatore desideri mantenere i duplicati (comportamento non relazionale ma spesso necessario a fini statistici o per risparmiare il gravoso costo computazionale dell'ordinamento), è necessario esplicitare la parola chiave **ALL** subito dopo l'operatore (es. **UNION ALL**, **EXCEPT ALL**).

Esempio 4.9 (Traduzione SQL: Squadre vincenti allo stadio Maradona). L'espressione algebrica $v_{casa} \cup v_{trasferta}$ vista nel primo esempio si traduce strutturando due query indipendenti collegate dall'operatore **UNION**:

```

1 SELECT squadra1 AS squadra_vincente
2 FROM partite
3 WHERE stadio = 'Maradona' AND goals1 > goals2
4 UNION
5 SELECT squadra2
6 FROM partite
7 WHERE stadio = 'Maradona' AND goals2 > goals1;

```

Nota Architetture: Lo schema della tabella virtuale risultante eredita sempre i nomi delle colonne dal **primo** blocco **SELECT** (valutato top-down). Pertanto, un'eventuale ridenominazione tramite alias (**AS squadra_vincente**) ha effetto solo se applicata nella query superiore; eventuali alias nel blocco inferiore verrebbero ignorati dal motore SQL.

4.7 Il Prodotto Cartesiano e le Giunzioni (Join)

Fino a questo momento abbiamo interrogato le relazioni in modo isolato (operazioni unarie) o abbiamo unito tabelle con struttura identica (operazioni insiemistiche). Tuttavia, la vera potenza del modello relazionale consiste nella capacità di combinare informazioni di natura diversa incrociando relazioni eterogenee. L'operatore primitivo che permette questa combinazione orizzontale è il prodotto cartesiano.

Il Prodotto Cartesiano Siano $r_1 \in R_1(X_1)$ e $r_2 \in R_2(X_2)$ due istanze di relazione. Il prodotto cartesiano $r_1 \times r_2$ è un operatore binario che concatena ogni singola tupla della prima relazione con ogni singola tupla della seconda.

- **Schema risultante (Grado):** Lo schema della nuova relazione è costituito dall'unione degli insiemi di attributi ($X_1 \cup X_2$). Se la prima relazione ha N_1 attributi e la seconda ne ha N_2 , il grado risultante è $N_1 + N_2$.
- **Impatto sulla cardinalità:** Il numero totale di tuple generate è il prodotto matematico delle cardinalità di partenza: $n_1 \cdot n_2$.

Osservazione 4.3 (L'esplosione combinatoria e l'assenza di semantica). Dal punto di vista ingegneristico, il prodotto cartesiano genera una rapida esplosione dimensionale. Se incrociamo una tabella di 1.000 studenti con una di 100 corsi, generiamo istantaneamente 100.000 tuple. Ancora più grave è il problema semantico: il prodotto cartesiano puro concatena tuple in modo incondizionato, associando (ad esempio) ogni studente a **tutti** gli esami esistenti, compresi quelli che non ha mai sostenuto. Genera dunque un'enorme quantità di informazioni prive di significato logico. Per questo motivo, l'operatore libero ($\times$) non viene quasi mai utilizzato direttamente, ma funge da base matematica per definire l'operatore di giunzione.

L'Operatore di Giunzione (Theta-Join) Per estrarre solo le tuple "coerenti" dal prodotto cartesiano, è necessario applicare una selezione che filtri le righe basandosi su un legame logico tra le due tabelle. Poiché questa combinazione è ubiquitaria nei database, l'algebra relazionale la fonde in un unico operatore derivato: la **Giunzione** (o Join), denotata dal simbolo a farfalla ($\bowtie$).

Per definizione, una giunzione condizionale (o Theta-Join) rispetto a un predicato p equivale a eseguire una selezione su un prodotto cartesiano:

$$r_1 \bowtie_p r_2 \equiv \sigma_p(r_1 \times r_2)$$

La condizione di giunzione è **quasi sempre un'uguaglianza** (Equi-Join) che lega la Primary Key (PK) di una tabella alla corrispondente Foreign Key (FK) dell'altra.

Esempio 4.10 (Equi-Join: Associazione corretta tra Studenti ed Esami). Consideriamo gli schemi $STUDENTE(matricola, nome, cognome)$ ed $ESAME(matr, corso, voto)$, dove $matr$ in $ESAME$ è chiave esterna verso $STUDENTE$. Per visualizzare l'elenco corretto degli esami sostenuti da ciascuno studente, applichiamo un Equi-Join sulla condizione di legame:

$$studente \bowtie_{matricola=matr} esame$$

Il risultato restituirà solo le tuple in cui il codice dello studente coincide, eliminando le migliaia di combinazioni errate generate dal prodotto cartesiano sottostante.

La Giunzione Naturale (Natural Join) La giunzione naturale è una variante del Join in cui **la condizione viene omessa** dal pedice dell'operatore. Il sistema applica automaticamente una condizione implicita e altera lo schema risultante in modo peculiare.

- **Condizione implicita:** Le due relazioni vengono giunte uguagliando i valori di **tutti gli attributi con lo stesso nome** (e stesso dominio) presenti in entrambi gli schemi. In termini insiemistici, la giunzione naturale opera un'intersezione sui valori delle colonne in comune.

- **Fusione delle colonne:** A differenza del Theta-Join (che duplica le colonne messe a confronto), la giunzione naturale **fonde gli attributi in comune** in un'unica colonna, evitando ridondanze nello schema risultante.
- **Assenza di attributi in comune:** Se le due tabelle non hanno alcun attributo omonimo, l'operatore non potendo applicare alcuna condizione "degenera" in un prodotto cartesiano incondizionato ($r_1 \bowtie r_2 \equiv r_1 \times r_2$).

Esempio 4.11 (Giunzione Naturale tramite Ridenominazione). Nell'esempio precedente, non possiamo eseguire un Natural Join diretto tra *studente* ed *esame* poiché l'attributo identificativo ha nomi diversi (*matricola* e *matr*). Possiamo però forzare la natura dell'operatore applicando prima una ridenominazione sull'istanza *esame*:

$$studente \bowtie \rho_{esame_mod(matricola, corso, voto)}(esame)$$

Essendo comparso un attributo omonimo (*matricola*), l'operatore eseguirà l'uguaglianza implicita e restituirà uno schema compatto: $R(matricola, nome, cognome, corso, voto)$.

Self-Join e l'uso strategico della Ridenominazione L'operazione di ridenominazione completa (ρ) si rivela indispensabile quando occorre confrontare tuple appartenenti alla **stessa** relazione. In algebra relazionale non è possibile eseguire $r \bowtie r$ senza creare un'insanabile ambiguità sugli attributi. È necessario generare una copia "virtuale" della relazione con un'intestazione diversa.

Esempio 4.12 (Cambiamenti di città). Siano dati gli schemi (dove PK è il primo attributo o la prima coppia):

- $PERSONA(cf, nome, cognome, dataN, dataM)$
- $RESIDENZA(cf, dataInizio, dataFine, città, via)$

Interrogazione: Estrarre Codice Fiscale, Nome e Cognome delle persone che hanno cambiato città di residenza nel corso del tempo.

Per risolvere l'interrogazione, dobbiamo cercare due tuple di residenza che abbiano lo stesso *cf* ma attributo *città* differente.

1. **Creazione della copia:** Rinominiamo la tabella residenza per avere due fattori indipendenti:

$$r_{old} = \rho_{r_old(cf', dI', dF', città', via')}(residenza)$$

2. **Self-Join e Filtraggio:** Uniamo l'istanza originale con la copia imponendo che la persona sia la stessa ($cf = cf'$), ed estraiamo con una selezione chi ha cambiato città:

$$cambi = \pi_{cf}(\sigma_{città \neq città'}(residenza \bowtie_{cf=cf'} r_{old}))$$

3. **Recupero anagrafica (Natural Join):** L'espressione *cambi* contiene solo il *cf*. Per ottenere Nome e Cognome, eseguiamo una giunzione naturale con l'anagrafica:

$$risultato = \pi_{cf, nome, cognome}(cambi \bowtie persona)$$

4.7.1 I Join in SQL

Lo standard SQL moderno (ANSI-92) implementa la giunzione separandola nettamente dalle normali condizioni di filtraggio (clausola **WHERE**), dedicando costrutti specifici direttamente nella clausola **FROM**.

Sintassi generale (Equi-Join):


```

1 SELECT <elenco_attributi>
2 FROM <tabella_1>
3 [INNER] JOIN <tabella_2> ON <condizione_di_giunzione>;

```

Q La parola chiave INNER

Scrivere JOIN oppure INNER JOIN ordina al DBMS di eseguire **esattamente la stessa operazione** (il Theta-Join o Equi-Join dell'algebra relazionale).

Il termine INNER (interno) ribadisce soltanto la natura restrittiva dell'operatore: le tuple che non soddisfano la condizione di giunzione vengono scartate. Poiché questo è il comportamento di default storico di SQL, la parola può essere omessa, ma viene spesso esplicitata nelle basi di codice di produzione per distinguere nettamente l'operazione dalle giunzioni esterne (OUTER JOIN).

La traduzione del nostro esempio originario ($studente \bowtie_{matricola=matr} esame$) si realizza applicando la notazione puntata per esplicitare la condizione tra Primary Key e Foreign Key:

```

1 SELECT s.matricola, s.nome, s.cognome, e.corso, e.voto
2 FROM studente AS s
3 JOIN esame AS e ON s.matricola = e.matr;

```

⚠ L'antiquata sintassi implicita (ANSI-89)

È possibile incontrare basi di dati o codice legacy in cui la giunzione viene espressa "alla vecchia maniera", eseguendo il prodotto cartesiano separando le tabelle con una virgola e inserendo la condizione di Join nel WHERE:

```

1 -- SINTASSI DEPRECATA (da evitare)
2 SELECT * FROM studente s, esame e WHERE s.matricola = e.matr
   ;

```

Questa prassi è fortemente sconsigliata: mescola pericolosamente la logica di legame strutturale tra tabelle con le normali condizioni di filtraggio (selezione σ), rendendo il codice pronò a omissioni che causano il famigerato *Cartesian Explosion*.

✓ Evitare la Giunzione Naturale in SQL

Sebbene la giunzione naturale ($\bowtie$) sia un operatore teorico fondamentale per semplificare le espressioni in algebra relazionale, la sua traduzione diretta in SQL (tramite la parola chiave NATURAL JOIN) è universalmente considerata un **anti-pattern ingegneristico** e non viene quasi mai utilizzata nel codice di produzione.

La regola d'oro nello sviluppo del software impone che **l'esplicito sia sempre meglio dell'implicito**. Pertanto, si utilizza pressoché esclusivamente la giunzione con condizione esplicita (JOIN ... ON ...) per tre ragioni architetturali critiche:

1. **Prevenzione delle evoluzioni occulte (Schema Evolution):** Se in futuro un amministratore aggiungesse a entrambe le tabelle una colonna di



servizio con lo stesso nome (es. `data_modifica` o `stato`), un `NATURAL JOIN` la ingloberebbe silenziosamente nella condizione di uguaglianza. La query inizierebbe a fallire o a omettere risultati senza mai generare un errore di compilazione.

2. **Prevenzione delle collisioni di naming:** Spesso chiavi primarie di entità diverse condividono lo stesso identificatore generico (es. `id`). La giunzione naturale le uguaglierebbe in automatico, estraendo dati matematicamente associati ma semanticamente spazzatura.
3. **Leggibilità del codice (Manutenibilità):** Esplicitare la clausola `ON` permette a chiunque legga la query di comprendere istantaneamente il legame strutturale tra le tabelle, senza essere costretto a interrogare la struttura fisica del database per capire quali colonne omonime il motore SQL abbia deciso di fondere.

Per questi motivi, la sintassi e l'uso del `NATURAL JOIN` vengono qui deliberatamente omessi a favore dell'approccio rigoroso basato sulle condizioni esplicite. Un paragrafo al riguardo è comunque presente in appendice per completezza.

4.8 Giunzioni Esterne (Outer Join)

L'operatore di giunzione interna (Inner Join) è intrinsecamente "distruttivo": se una tupla della prima relazione non trova alcuna corrispondenza nella seconda relazione rispetto alla condizione imposta, viene inesorabilmente scartata dal risultato finale.

In molti scenari informativi, tuttavia, è essenziale preservare traccia delle entità "orfane" (es. gli studenti che non hanno ancora sostenuto alcun esame, o i clienti che non hanno effettuato ordini). Per rispondere a questa esigenza, l'algebra relazionale introduce le **Giunzioni Esterne** (Outer Join).

Le giunzioni esterne estendono il risultato di una normale giunzione recuperando le tuple scartate e "riempiendo" (padding) gli attributi mancanti della relazione opposta con il marcatore di incognita `NULL`.

Esistono tre varianti di questo operatore (anche per esse vale il divieto architetturale di omettere la condizione a favore della variante "naturale"):

- **Left Outer Join** ($r_1 \bowtie\!\!\!\bowtie r_2$): Preserva **tutte** le tuple della relazione di sinistra (r_1). Se una tupla di r_1 non ha corrispondenze in r_2 , viene inclusa nel risultato assegnando `NULL` a tutti gli attributi provenienti da r_2 .
- **Right Outer Join** ($r_1 \bowtie\!\!\!\bowtie r_2$): Preserva **tutte** le tuple della relazione di destra (r_2), inserendo `NULL` negli attributi di r_1 per le tuple orfane.
- **Full Outer Join** ($r_1 \bowtie\!\!\!\bowtie r_2$): È l'unione dei due comportamenti. Preserva integralmente entrambe le relazioni, inserendo `NULL` a destra o a sinistra ove manchi la corrispondenza.

Esempio 4.13 (Left Join: Studenti con e senza esami). Siano dati gli schemi:

STUDENTE(*matricola*, *nome*), *ESAME*(*matr*, *corso*, *voto*),

con le seguenti piccole istanze:

- *studente*: {(101, 'Mario'), (102, 'Luigi')}
- *esame*: {(101, 'Basi di Dati', 30)}

Eseguendo una giunzione esterna sinistra:

studente $\bowtie\!\!\!\bowtie_{matricola=matr}$ *esame*

La tupla di Mario trova corrispondenza e si fonde regolarmente. La tupla di Luigi non trova corrispondenza in *esame*, ma viene "salvata" dall'operatore *Left*, generando la riga: (102, 'Luigi', NULL, NULL, NULL).

4.8.1 Le Giunzioni Esterne in SQL

Il linguaggio SQL implementa questi operatori estendendo la sintassi della clausola JOIN. Esattamente come per INNER, la parola chiave OUTER è considerata ridondante e facoltativa dallo standard (es. scrivere LEFT JOIN equivale a tutti gli effetti a scrivere LEFT OUTER JOIN).

Sintassi generale (Left Join):

```
1 SELECT s.matricola, s.nome, e.corso, e.voto
2 FROM studente AS s
3 LEFT JOIN esame AS e ON s.matricola = e.matr;
```

✓ Realtà di Produzione: Il tramonto del RIGHT JOIN

Sebbene il RIGHT JOIN esista e sia formalmente corretto, nel codice di produzione moderno è universalmente evitato. Per convenzione di leggibilità *top-down* (dall'alto verso il basso), gli ingegneri preferiscono posizionare la tabella principale nella clausola FROM ed eseguire LEFT JOIN a cascata, piuttosto che invertire mentalmente l'ordine di lettura usando giunzioni destre. Un RIGHT JOIN può (e deve) sempre essere convertito in un LEFT JOIN semplicemente scambiando l'ordine delle tabelle nel codice.

Osservazione 4.4 (La natura della FULL JOIN). A causa della scarsa utilità pratica di estrarre orfani bidirezionali in modelli dati strutturati, la FULL JOIN è così rara che molti DBMS commerciali (come MySQL o SQLite) non la implementano nativamente. Per ottenerla, si ricorre all'algebra insiemistica: l'unione (UNION) tra una giunzione sinistra (A LEFT JOIN B) e la sua inversa (B LEFT JOIN A).

4.8.2 Il pattern "Anti-Join": Isolare le assenze

L'utilità strategica più importante delle giunzioni esterne non risiede solo nel "mostrare tutto", ma nel fornire uno strumento per **isolare matematicamente le anomalie o le mancanze**.

Per rispondere all'interrogazione "*Trovare gli studenti che **non** hanno sostenuto alcun esame*", l'approccio più robusto consiste nel combinare un Outer Join con la gestione della logica ternaria dei valori NULL studiata nei capitoli precedenti.

1. Si esegue un LEFT JOIN per estrarre tutti gli studenti, portando a bordo i NULL per chi non ha esami. 2. Si applica una selezione (WHERE) filtrando deliberatamente **solo** le righe in cui la Primary Key della tabella agganciata risulta NULL.

Questo costrutto ingegneristico è noto come **Anti-Join**. È enormemente più efficiente rispetto all'uso di operazioni insiemistiche (EXCEPT) perché evita al motore del database di dover eseguire onerosi ordinamenti per la de-duplicazione.

```
1 -- Estrae solo gli studenti SENZA esami
2 SELECT s.matricola, s.nome
3 FROM studente AS s
```

```
4 LEFT JOIN esame AS e ON s.matricola = e.matr  
5 WHERE e.matr IS NULL;
```

Bozze

A.1 parti di lezioni da rivedere

- esercitazione su class diagrams: prima parte lezione 6.
- esercitazione su schemi relazionali e ristrutturazione logica del class diagram: seconda parte lezione 12.
- primi 30 minuti lezione 13: riepilogo e precisazioni rilevanti.
- esercizi assegnati sulle prime query fine lezione 15

A.2 La Giunzione Naturale in SQL: NATURAL JOIN e USING (da inserire in appendice)

A differenza dell'algebra relazionale dove l'omissione della condizione trasforma implicitamente un Theta-Join in un Natural Join ($\bowtie$), il linguaggio SQL richiede l'uso di parole chiave esplicite per attivare questo comportamento ed evitare prodotti cartesiani accidentali.

Esistono due approcci sintattici per implementare la fusione delle colonne omonime tipica della giunzione naturale:

L'operatore puro: NATURAL JOIN Questa è la traduzione letterale dell'operatore matematico $\bowtie$.

```
1 SELECT *
2 FROM tabella_1
3 NATURAL JOIN tabella_2;
```

Quando il motore SQL incontra `NATURAL JOIN`, esegue autonomamente due operazioni:

1. **Ricerca degli attributi:** Ispeziona gli schemi delle due relazioni alla ricerca di tutte le colonne aventi **esattamente lo stesso nome**.
2. **Condizione e Fusione:** Genera un'uguaglianza implicita su tali colonne (es. `t1.id = t2.id`) e, nel *Result Set* finale, **fonde** le colonne omonime in una sola, evitando ridondanze.

⚠ La fragilità strutturale del NATURAL JOIN

Nonostante la sua eleganza matematica, l'uso di `NATURAL JOIN` in produzione è universalmente considerato un **anti-pattern ingegneristico**. Il motivo risiede nella sua dipendenza occulta e non controllata dai nomi delle colonne. Se mesi dopo la stesura del codice un amministratore aggiungesse a entrambe le tabelle una colonna di servizio generica (es. `note` o `data_inserimento`), il DBMS modificherebbe silenziosamente la condizione di giunzione pretendendo l'uguaglianza anche su queste nuove colonne (`AND t1.note = t2.note`). Questo distruggerebbe la logica dell'interrogazione originando risultati errati o insiemi vuoti, senza lanciare alcun errore di compilazione.

L'alternativa ingegneristica: La clausola USING Per ovviare alla pericolosità del NATURAL JOIN preservandone però il vantaggio sintattico (la fusione delle colonne omonime), lo standard SQL ha introdotto la clausola **USING**.

Questa clausola va a sostituire **ON** e permette al progettista di dichiarare esplicitamente su quali colonne omonime deve avvenire la giunzione, rendendo il codice immune a future modifiche dello schema.

Sintassi con USING:

```
1  -- Supponendo che entrambe le tabelle abbiano un attributo '
   matricola '
2  SELECT *
3  FROM studente
4  JOIN esame USING (matricola);
```

A differenza della classica clausola **ON** (che restituirebbe due colonne distinte: **studente.matricola** ed **esame.matricola**), la clausola **USING** istruisce il database a **fondere** l'attributo in una singola colonna, rispecchiando in totale sicurezza il comportamento algebrico della giunzione naturale.